



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Análisis comparativo de
arquitecturas de redes
neuronales convolucionales
para la detección de pose, la
clasificación y evaluación de
enfermedades
neurodegenerativas, como la
esclerosis lateral amiotrófica,
en imágenes 2D.**



Presentado por Samuel Murillo Arce
en Universidad de Burgos — 7 de julio de 2026
Tutor: Fernando Rivas Navazo



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



Fernando Rivas Navazo, profesor del departamento de Ingeniería Electromecánica, área de Tecnología Electrónica.

Expone:

Que el alumno D. Samuel Murillo Arce, con DNI 71791802Q, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado título de TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 7 de julio de 2026

Vº. Bº. del Tutor:

Fernando Rivas Navazo

Resumen

En las últimas décadas, el aumento de la esperanza de vida y el envejecimiento progresivo de la población han incrementado la incidencia de enfermedades crónicas y neurodegenerativas. Entre ellas, la esclerosis lateral amiotrófica (ELA) constituye una patología de especial relevancia por su carácter progresivo, su impacto sobre la movilidad y la ausencia de un tratamiento curativo definitivo. La ELA afecta principalmente a las neuronas motoras responsables del control de la musculatura voluntaria, provocando una pérdida gradual de fuerza, alteraciones en la coordinación motora y dificultades crecientes para realizar actividades cotidianas.

A medida que la enfermedad avanza, los pacientes pueden experimentar debilidad muscular, espasticidad, fatiga, pérdida de precisión en los movimientos, alteraciones en la marcha, problemas posturales y limitaciones funcionales cada vez más acusadas. Estas manifestaciones hacen necesario un seguimiento clínico y terapéutico continuado, en el que la evaluación del movimiento desempeña un papel fundamental. Sin embargo, muchas de estas valoraciones requieren la presencia de profesionales especializados, instrumentación específica o sesiones presenciales, lo que puede limitar la frecuencia del seguimiento y dificultar una monitorización objetiva y accesible.

En este contexto, las técnicas de visión por computador y aprendizaje profundo ofrecen nuevas posibilidades para el análisis automatizado del movimiento humano. Los modelos de detección de pose permiten identificar puntos anatómicos relevantes del cuerpo a partir de imágenes o vídeos, generando una representación estructurada del esqueleto humano. A partir de dicha información es posible extraer variables biomecánicas, analizar la evolución de los movimientos y detectar desviaciones o patrones anómalos que puedan resultar útiles en entornos clínicos, terapéuticos o de rehabilitación.

Este Trabajo de Fin de Grado se centra en el estudio y comparación de distintos modelos de detección de pose basados en redes neuronales, con especial atención a su aplicación en vídeos médicos y terapéuticos relacionados con alteraciones motoras asociadas a la ELA. El objetivo principal es analizar la viabilidad de estos modelos para extraer esqueletos humanos a partir de vídeos 2D, evaluar su comportamiento en diferentes condiciones y explorar su utilidad para el seguimiento del movimiento en ejercicios de rehabilitación o valoración funcional.

La metodología propuesta parte del procesamiento de vídeos previamente existentes, obtenidos de bases de datos médicas, terapéuticas o de rehabilitación. Dichos vídeos se descomponen en secuencias de frames, sobre los que se aplican modelos de estimación de pose para localizar puntos clave del cuerpo humano. Posteriormente, las coordenadas obtenidas se normalizan y se analizan con el fin de extraer información relevante, como ángulos articulares, rangos de movimiento, simetría corporal, estabilidad postural, continuidad temporal y posibles anomalías en la ejecución de los movimientos.

Además del análisis comparativo de los modelos, el trabajo contempla el diseño de una aplicación de escritorio con interfaz gráfica que permita cargar vídeos, seleccionar el modelo de detección de pose, visualizar el esqueleto superpuesto sobre la imagen, consultar métricas de movimiento y exportar los resultados obtenidos. Esta herramienta pretende ofrecer un entorno sólido y accesible para apoyar el análisis de ejercicios terapéuticos, facilitando la interpretación de los datos tanto desde una perspectiva técnica como funcional.

La solución planteada no pretende sustituir el diagnóstico ni la valoración realizada por profesionales sanitarios, sino servir como herramienta de apoyo para el análisis objetivo del movimiento. Su aplicación podría contribuir al seguimiento de pacientes con ELA y otras patologías neuromotoras, así como a procesos de rehabilitación física en los que sea necesario evaluar la calidad, simetría o evolución de determinados ejercicios. De este modo, el proyecto se sitúa en una línea de trabajo orientada a la automatización de la evaluación motora, la mejora del seguimiento terapéutico y el desarrollo de tecnologías accesibles para entornos clínicos y domiciliarios.

Descriptores

Esclerosis lateral amiotrófica, ELA, enfermedades neurodegenerativas, detección de pose humana, estimación de pose, visión por computador, redes neuronales convolucionales, aprendizaje profundo, análisis de movimiento, vídeos terapéuticos, rehabilitación, esqueletos humanos, keypoints, anomalías posturales. . . .

Abstract

In recent decades, increased life expectancy and the progressive ageing of the population have led to a rise in the incidence of chronic and neurodegenerative diseases. Among them, amyotrophic lateral sclerosis (ALS) is particularly relevant due to its progressive nature, its impact on mobility and the absence of a definitive curative treatment. ALS mainly affects the motor neurons responsible for controlling voluntary muscles, causing a gradual loss of strength, impaired motor coordination and increasing difficulties in performing daily activities.

As the disease progresses, patients may experience muscle weakness, spasticity, fatigue, reduced movement precision, gait alterations, postural problems and increasingly severe functional limitations. These symptoms make continuous clinical and therapeutic monitoring necessary, where movement assessment plays a key role. However, many of these evaluations require the presence of specialised professionals, specific equipment or in-person sessions, which may limit the frequency of follow-up and make objective and accessible monitoring more difficult.

In this context, computer vision and deep learning techniques offer new possibilities for the automated analysis of human movement. Pose detection models make it possible to identify relevant anatomical points of the body from images or videos, generating a structured representation of the human skeleton. From this information, it is possible to extract biomechanical variables, analyse movement evolution and detect deviations or anomalous patterns that may be useful in clinical, therapeutic or rehabilitation environments.

This Bachelor's Degree Final Project focuses on the study and comparison of different neural-network-based pose detection models, with particular attention to their application in medical and therapeutic videos related to motor impairments associated with ALS. The main objective is to analyse the feasibility of these models for extracting human skeletons from 2D videos, evaluate their behaviour under different conditions and explore their usefulness for movement monitoring in rehabilitation exercises or functional assessment.

The proposed methodology starts from the processing of pre-existing videos obtained from medical, therapeutic or rehabilitation databases. These videos are decomposed into sequences of frames, on which pose estimation models are applied to locate key points of the human body. Subsequently, the obtained coordinates are normalised and analysed in order to extract relevant information, such as joint

angles, ranges of motion, body symmetry, postural stability, temporal continuity and possible anomalies in movement execution.

In addition to the comparative analysis of the models, the project includes the design of a desktop application with a graphical user interface that allows users to load videos, select the pose detection model, visualise the skeleton overlaid on the image, consult movement metrics and export the obtained results. This tool aims to provide a robust and accessible environment to support the analysis of therapeutic exercises, facilitating data interpretation from both a technical and functional perspective.

The proposed solution is not intended to replace diagnosis or assessment performed by healthcare professionals, but rather to serve as a support tool for the objective analysis of movement. Its application could contribute to the monitoring of patients with ALS and other neuromotor disorders, as well as to physical rehabilitation processes in which it is necessary to evaluate the quality, symmetry or evolution of specific exercises. In this way, the project is framed within a line of work aimed at automating motor assessment, improving therapeutic follow-up and developing accessible technologies for clinical and home-based environments.

Keywords

Amyotrophic lateral sclerosis, ALS, neurodegenerative diseases, human pose detection, pose estimation, computer vision, convolutional neural networks, deep learning, movement analysis, therapeutic videos, rehabilitation, human skeletons, keypoints, postural anomalies

Índice general

Índice general	v
Índice de figuras	vii
Índice de tablas	viii
1. Introducción	1
1.1. Estructura de la memoria	3
1.2. Materiales entregados	6
2. Objetivos del proyecto	9
2.1. Objetivo general	10
2.2. Objetivos específicos	11
2.3. Objetivos derivados de los requisitos del software	16
2.4. Objetivos técnicos del proyecto	19
2.5. Alcance del proyecto	21
3. Conceptos teóricos	23
3.1. Esclerosis lateral amiotrófica	23
3.2. Visión por computador y aprendizaje profundo	24
3.3. Redes neuronales convolucionales	25
3.4. Detección de pose humana	25
3.5. Modelos actuales de detección de pose	26
3.6. Representación de la pose mediante keypoints	27
3.7. Evaluación técnica de modelos de detección de pose	28
3.8. Anomalías en la estimación de esqueletos	29
3.9. Aplicación del análisis de pose en vídeos terapéuticos	30

4. Técnicas y herramientas	31
4.1. Metodología de desarrollo	31
4.2. Lenguaje de programación	32
4.3. Entorno de desarrollo	33
4.4. Procesamiento de vídeo con OpenCV	34
4.5. Modelos de detección de pose considerados	35
4.6. Interfaz gráfica de usuario	36
4.7. Análisis y almacenamiento de datos	37
4.8. Visualización de resultados	38
4.9. Empaquetado y distribución de la aplicación	39
4.10. Comparativa y justificación de herramientas seleccionadas	39
5. Aspectos relevantes del desarrollo del proyecto	41
5.1. Introducción	41
5.2. Metodología de desarrollo	42
5.3. Análisis del problema	42
5.4. Diseño de la arquitectura software	43
5.5. Diseño de la interfaz gráfica	44
5.6. Implementación del procesamiento de vídeo	46
5.7. Integración de los modelos de estimación de pose	47
5.8. Procesamiento y análisis de los keypoints	49
5.9. Exportación de resultados	49
5.10. Dificultades encontradas durante el desarrollo	50
5.11. Valoración del desarrollo realizado	57
6. Trabajos relacionados	59
6.1. Estimación de pose humana mediante aprendizaje profundo	59
6.2. Modelos modernos orientados a eficiencia e integración	60
6.3. Análisis de movimiento y aplicaciones terapéuticas	61
6.4. Relación con el proyecto desarrollado	62
7. Conclusiones y Líneas de trabajo futuras	63
7.1. Conclusiones	63
7.2. Líneas de trabajo futuras	65
Bibliografía	67

Índice de figuras

3.1. Flujo general de procesamiento de vídeo mediante técnicas de visión por computador. Fuente: elaboración propia.	24
3.2. Esquema simplificado de una red neuronal convolucional aplicada al análisis de imágenes. Fuente: elaboración propia.	25
3.3. Comparación conceptual entre enfoques <i>top-down</i> y <i>bottom-up</i> en detección de pose humana. Fuente: elaboración propia.	26
3.4. Representación esquemática de una pose humana mediante puntos corporales y conexiones entre articulaciones. Fuente: elaboración propia.	28
3.5. Ejemplos de posibles inconsistencias técnicas en la estimación de keypoints entre frames consecutivos. Fuente: elaboración propia.	29
5.1. Interfaz principal de PoseLab.	46

Índice de tablas

4.1. Resumen de alternativas consideradas y herramientas seleccionadas para el proyecto.	40
--	----

1. Introducción

El presente Trabajo de Fin de Grado se centra en el estudio, análisis y comparación de modelos de detección de pose humana basados en técnicas de visión por computador y aprendizaje profundo, con especial atención a su aplicación sobre vídeos médicos, terapéuticos o de rehabilitación relacionados con alteraciones motoras, como las asociadas a la esclerosis lateral amiotrófica (ELA). Aunque el contexto de aplicación se sitúa en el ámbito de las enfermedades neuromotoras, el enfoque principal del trabajo es de carácter informático, orientado a evaluar el comportamiento técnico de distintos modelos de estimación de pose y la calidad de los esqueletos generados a partir de vídeos 2D.

La ELA es una enfermedad neurodegenerativa que afecta progresivamente a las neuronas motoras responsables del control de la musculatura voluntaria. Como consecuencia, las personas que la padecen pueden experimentar pérdida de fuerza, reducción de movilidad, alteraciones en la coordinación, dificultad para realizar movimientos precisos y limitaciones funcionales cada vez mayores. Este tipo de manifestaciones hace que los vídeos terapéuticos o de seguimiento motor sean una fuente de información especialmente interesante para analizar el movimiento humano. No obstante, este trabajo no pretende realizar diagnóstico médico ni valorar clínicamente la evolución de un paciente, sino estudiar cómo diferentes modelos computacionales son capaces de detectar y representar de forma precisa la pose humana en este tipo de secuencias.

En los últimos años, los avances en inteligencia artificial han impulsado el desarrollo de sistemas capaces de interpretar imágenes y vídeos de forma automática. Dentro de este campo, la detección de pose humana permite localizar puntos anatómicos clave del cuerpo, como hombros, codos, muñecas, caderas, rodillas o tobillos, generando una representación estructurada del

esqueleto humano. A partir de estos puntos, también llamados keypoints, es posible analizar la posición del cuerpo en cada frame y estudiar la coherencia temporal de la detección a lo largo del vídeo. Esta información resulta especialmente útil desde el punto de vista informático, ya que permite comparar modelos en términos de precisión, estabilidad, robustez, continuidad temporal y fiabilidad de las coordenadas estimadas.

El trabajo plantea una aproximación técnica orientada a estudiar distintos modelos de estimación de pose, valorar sus características principales y analizar su adecuación al procesamiento de vídeos 2D. Entre los modelos considerados se encuentran alternativas ampliamente utilizadas en visión por computador, como MediaPipe Pose, MoveNet, YOLO-Pose, OpenPose, AlphaPose, HRNet, RTMPose o ViTPose. Cada uno de ellos presenta diferencias en cuanto a arquitectura, número de puntos detectados, formato de salida, velocidad de inferencia, precisión, facilidad de integración, dependencia de recursos hardware y comportamiento ante condiciones complejas, como oclusiones, cambios de iluminación, movimientos rápidos o pérdida parcial del cuerpo en la imagen.

Además de comparar las características generales de cada modelo, el proyecto contempla el análisis de anomalías en los esqueletos generados por los propios sistemas de detección de pose. En este contexto, una anomalía no se interpreta como una alteración física o médica del sujeto, sino como un posible error, desviación o inconsistencia producida por el modelo durante la estimación de los keypoints. Algunos ejemplos de estas anomalías pueden ser desplazamientos bruscos e injustificados de un punto entre frames consecutivos, detecciones con baja confianza, pérdida temporal de articulaciones, intercambio de puntos entre el lado izquierdo y derecho del cuerpo, posiciones anatómicamente incoherentes o variaciones excesivas en la longitud de los segmentos del esqueleto. El análisis de este tipo de errores permite evaluar la estabilidad y la calidad de los modelos, así como determinar cuáles resultan más adecuados para su uso en aplicaciones reales.

Desde esta perspectiva, el proyecto no se limita a superponer un esqueleto sobre un vídeo, sino que busca estudiar de forma crítica la fiabilidad de los modelos utilizados. Para ello, se plantea la extracción y normalización de los keypoints obtenidos, el cálculo de métricas relacionadas con la calidad de la detección y la identificación de comportamientos anómalos en las salidas generadas. Estas métricas pueden incluir la confianza media de los puntos, el porcentaje de frames con detección completa, la variación temporal de las coordenadas, la estabilidad de las distancias entre articulaciones, la suavidad de las trayectorias y la coherencia espacial del esqueleto. De este modo, la

detección de anomalías se convierte en un mecanismo de evaluación técnica del rendimiento del modelo, y no en una herramienta de valoración clínica del paciente.

El objetivo final del proyecto es sentar las bases para el desarrollo de una aplicación de escritorio con interfaz gráfica que permita cargar vídeos médicos o terapéuticos, aplicar distintos modelos de detección de pose, visualizar los esqueletos obtenidos, analizar la calidad de las detecciones y exportar los resultados. La intención es que el programa tenga una apariencia sólida y profesional, y que funcione como una herramienta informática para comparar modelos, revisar visualmente sus salidas y detectar posibles inconsistencias en los esqueletos generados. Aunque el ámbito de prueba se relacione con vídeos de carácter médico o terapéutico, el sistema se plantea como una aplicación de análisis técnico de modelos de visión por computador, no como una solución clínica.

La elección de este enfoque responde a la necesidad de explorar tecnologías accesibles para el procesamiento automático de vídeo y la estimación de pose humana. En contextos relacionados con la ELA u otras alteraciones neuromotoras, los movimientos pueden presentar características complejas para los modelos de visión artificial, como lentitud, pérdida de amplitud, asimetría, apoyo de dispositivos externos, cambios posturales o aparición de oclusiones. Estas condiciones suponen un reto desde el punto de vista computacional y permiten estudiar hasta qué punto los modelos de detección de pose mantienen una salida estable, precisa y coherente. Por tanto, el valor principal del trabajo reside en analizar la robustez de diferentes modelos ante vídeos reales o semirrealistas, identificando sus ventajas, limitaciones y posibles errores de estimación.

1.1. Estructura de la memoria

La memoria se organiza en varios capítulos que permiten abordar el proyecto de forma progresiva, desde la contextualización del problema hasta el diseño del sistema y las conclusiones obtenidas.

En primer lugar, se presenta una introducción general al trabajo, donde se expone el contexto del proyecto, la motivación principal, el problema que se pretende abordar y la orientación general de la solución propuesta. Esta sección sitúa al lector en el ámbito de la visión por computador, la detección de pose humana y el análisis de vídeos relacionados con entornos médicos o terapéuticos, aclarando que el objetivo fundamental del proyecto es evaluar modelos informáticos y no realizar diagnóstico clínico.

A continuación, se desarrolla el apartado de objetivos, en el que se define el propósito principal del Trabajo de Fin de Grado y se detallan los objetivos específicos que guían su desarrollo. Estos objetivos incluyen el estudio de modelos de detección de pose, la búsqueda de bases de datos adecuadas, la comparación técnica de diferentes aproximaciones, la definición de métricas para evaluar la calidad de los esqueletos generados, la detección de anomalías producidas por los modelos y el diseño de una aplicación que permita aplicar estos análisis sobre vídeos reales.

Posteriormente, se incluye un capítulo dedicado al marco teórico, donde se explican los conceptos necesarios para comprender el resto del trabajo. En este bloque se abordan aspectos relacionados con la esclerosis lateral amiotrófica como contexto de aplicación, la visión por computador, las redes neuronales convolucionales, el aprendizaje profundo, la estimación de pose humana, la representación mediante keypoints, la normalización de esqueletos y los criterios de evaluación de modelos aplicados a secuencias de vídeo.

El siguiente bloque corresponde al estado del arte, donde se revisan los principales modelos de detección de pose humana existentes. En esta sección se estudian modelos como MediaPipe Pose, MoveNet, YOLO-Pose, OpenPose, AlphaPose, HRNet, RTMPose y ViTPose, analizando sus características, ventajas, limitaciones y posibles aplicaciones en vídeos 2D. También se revisan aproximaciones relacionadas con la evaluación de la calidad de las detecciones, la estabilidad temporal de los keypoints y la detección de inconsistencias en esqueletos generados automáticamente.

Después se presenta el capítulo de búsqueda y análisis de datasets, en el que se recopilan y valoran bases de datos públicas que puedan resultar útiles para el proyecto. Se consideran datasets generales de estimación de pose, conjuntos de datos relacionados con rehabilitación, vídeos terapéuticos, marcha humana, ejercicios físicos y, cuando sea posible, recursos vinculados a enfermedades neuromotoras o neurodegenerativas como la ELA. Este apartado resulta especialmente importante, ya que la disponibilidad y calidad de los vídeos condiciona la comparación entre modelos y la detección de errores o desviaciones en sus salidas.

Una vez definidos los modelos y los datos, la memoria aborda la metodología del proyecto. En este capítulo se describe el flujo de trabajo previsto: selección de vídeos, extracción de frames, aplicación de modelos de detección de pose, obtención de keypoints, normalización de coordenadas, cálculo de métricas de calidad y análisis de posibles anomalías en los esqueletos generados. También se explican los criterios de comparación entre modelos,

como velocidad, estabilidad, completitud de detección, confianza media, coherencia anatómica, robustez ante oclusiones y facilidad de integración.

Más adelante, se incluye un capítulo dedicado al diseño del sistema, donde se describe la arquitectura general de la aplicación propuesta. Se detallan los módulos principales del programa, como la interfaz gráfica, el gestor de vídeo, el selector de modelos, el procesador de keypoints, el módulo de métricas de calidad, el sistema de detección de anomalías en las salidas del modelo y las funciones de exportación. Este apartado permite justificar la organización interna del software y demostrar que el desarrollo se plantea de forma modular, escalable y mantenible.

En el capítulo de implementación, se describirá el desarrollo técnico de la aplicación de escritorio. Aunque la programación se abordará en una fase posterior del proyecto, la memoria recogerá las decisiones tecnológicas adoptadas, como el uso de Python, OpenCV para el procesamiento de vídeo, librerías de detección de pose y un framework gráfico como PySide6 o PyQt. También se explicará cómo se integran los modelos seleccionados, cómo se normalizan sus salidas y cómo se presentan al usuario los esqueletos, métricas e inconsistencias detectadas.

El apartado de pruebas y resultados recogerá los experimentos realizados con los vídeos seleccionados. En esta sección se comparará el comportamiento de los modelos aplicados, se analizarán las métricas obtenidas y se estudiará la calidad de los esqueletos generados. También se evaluará la capacidad del sistema para identificar errores o anomalías en las salidas de los modelos, como saltos bruscos de keypoints, pérdida de articulaciones, baja confianza, deformaciones del esqueleto o inconsistencias temporales.

Finalmente, la memoria concluirá con un capítulo de conclusiones y líneas futuras, donde se resumirán los principales resultados del trabajo, se valorará el grado de cumplimiento de los objetivos y se identificarán posibles mejoras. Entre las líneas futuras podrían incluirse la incorporación de nuevos modelos, la validación con más datasets, el uso de técnicas avanzadas para detectar inconsistencias temporales, la comparación con anotaciones manuales, la mejora del filtrado de keypoints o la adaptación del sistema a otros tipos de vídeos.

1.2. Materiales entregados

Además de la memoria principal, el Trabajo de Fin de Grado irá acompañado de una serie de materiales complementarios que permitirán documentar, justificar y reproducir el desarrollo realizado.

En primer lugar, se incluirán anexos técnicos, donde se recogerá información adicional que no convenga introducir de forma extensa en el cuerpo principal de la memoria. Estos anexos podrán contener tablas comparativas completas de los modelos estudiados, fichas técnicas de cada arquitectura, detalles sobre los datasets localizados, ejemplos de métricas de calidad, configuraciones utilizadas en las pruebas, criterios de detección de anomalías en los esqueletos generados y capturas de pantalla de la aplicación.

También se podrá incluir un manual de instalación, cuyo objetivo será explicar los pasos necesarios para preparar el entorno de ejecución del programa. En este documento se especificarán las herramientas necesarias, las dependencias utilizadas, la versión de Python recomendada y las instrucciones básicas para ejecutar la aplicación correctamente.

Otro material relevante será el manual de usuario, orientado a explicar el funcionamiento de la aplicación desde el punto de vista de una persona que quiera utilizarla. Este manual describirá cómo cargar un vídeo, seleccionar un modelo de detección de pose, iniciar el procesamiento, interpretar la visualización del esqueleto, revisar las métricas de calidad, identificar posibles inconsistencias detectadas por el sistema y exportar los resultados generados.

Asimismo, se entregará el código fuente del programa, organizado de forma clara y modular. La estructura del código deberá reflejar las distintas partes del sistema, separando la interfaz gráfica, la gestión de vídeo, la integración de modelos, el procesamiento de keypoints, el cálculo de métricas de calidad, la detección de anomalías en las salidas y la generación de archivos exportables. Esta organización facilitará tanto la revisión académica como futuras ampliaciones del proyecto.

Por último, podrán incluirse archivos de prueba o ejemplos de salida, como vídeos procesados, ficheros CSV o JSON con coordenadas de keypoints, gráficas de estabilidad temporal, capturas de esqueletos generados, tablas comparativas de rendimiento y ejemplos de informes producidos por la aplicación. Estos materiales servirán para demostrar el funcionamiento del sistema y apoyar la evaluación de los resultados obtenidos.

En conjunto, la memoria, los anexos, el código fuente y los materiales complementarios permitirán presentar un proyecto completo, coherente y

reproducible, en el que se combinen fundamentos teóricos, análisis comparativo de modelos, diseño software y aplicación práctica al estudio técnico de modelos de detección de pose en vídeos 2D relacionados con contextos terapéuticos o neuromotores.

El repositorio del proyecto se puede encontrar en **GitHub**¹.

¹https://github.com/SamuelMurilloArce/TRABAJO_FIN_GRADO

2. Objetivos del proyecto

Este capítulo tiene como finalidad definir de manera clara los objetivos que se persiguen con el desarrollo del presente Trabajo de Fin de Grado. La definición de estos objetivos resulta fundamental, ya que permite delimitar el alcance del proyecto, establecer qué se pretende conseguir y diferenciar entre las metas relacionadas con el estudio técnico de los modelos de detección de pose y aquellas vinculadas al diseño de la aplicación software.

El proyecto se sitúa dentro del ámbito de la visión por computador, el aprendizaje profundo y el análisis automático de vídeo. Su finalidad principal no es realizar un diagnóstico médico ni sustituir la valoración de profesionales sanitarios, sino estudiar el comportamiento de distintos modelos de detección de pose humana cuando son aplicados a vídeos 2D de carácter médico, terapéutico o de rehabilitación, con especial atención a contextos relacionados con alteraciones neuromotoras como la esclerosis lateral amiotrófica. Desde esta perspectiva, el trabajo se plantea como un proyecto eminentemente informático, orientado a analizar modelos, procesar datos visuales, comparar resultados y diseñar una herramienta capaz de facilitar dicha evaluación.

La detección de pose humana permite estimar la localización de distintos puntos anatómicos del cuerpo, conocidos habitualmente como keypoints, a partir de imágenes o secuencias de vídeo. Estos puntos permiten construir una representación simplificada del esqueleto humano y constituyen la base sobre la que se pueden realizar análisis posteriores. Sin embargo, no todos los modelos ofrecen el mismo nivel de precisión, estabilidad o robustez. En función de la arquitectura utilizada, del número de puntos detectados, del formato de salida, de la calidad del vídeo o de las condiciones de la escena, los resultados pueden variar de forma significativa. Por ello, uno de los

propósitos centrales del proyecto consiste en estudiar estas diferencias y valorar qué modelos resultan más adecuados para su integración en una aplicación práctica.

Asimismo, el trabajo incorpora como objetivo relevante el análisis de anomalías en los esqueletos generados por los modelos. En este caso, el concepto de anomalía se entiende desde un punto de vista técnico: no se refiere a una anomalía corporal o médica del sujeto que aparece en el vídeo, sino a posibles errores o inconsistencias en la salida producida por el modelo de detección de pose. Entre estas anomalías pueden encontrarse desplazamientos bruscos de puntos entre frames consecutivos, pérdida temporal de articulaciones, detecciones con baja confianza, intercambio entre puntos del lado izquierdo y derecho del cuerpo, deformaciones artificiales del esqueleto o variaciones incoherentes en la longitud de los segmentos corporales estimados. La detección de este tipo de comportamientos permite valorar la fiabilidad del modelo y estudiar hasta qué punto sus resultados son útiles para un análisis posterior.

A partir de esta idea general, los objetivos del proyecto pueden organizarse en diferentes niveles. En primer lugar, se establece un objetivo general que resume la finalidad principal del trabajo. Posteriormente, se definen una serie de objetivos específicos vinculados al estudio de modelos, la búsqueda de datos, el análisis técnico de las salidas generadas y el diseño del sistema software. Finalmente, se distinguen los objetivos derivados de los requisitos de la aplicación que se pretende construir y los objetivos técnicos asociados a la puesta en práctica del proyecto.

2.1. Objetivo general

El objetivo general de este Trabajo de Fin de Grado es analizar, comparar y evaluar distintos modelos de detección de pose humana basados en técnicas de aprendizaje profundo, aplicados a vídeos 2D de carácter médico, terapéutico o de rehabilitación, con el fin de estudiar la calidad, estabilidad y fiabilidad de los esqueletos generados y sentar las bases para el desarrollo de una aplicación de escritorio que permita visualizar, analizar y exportar los resultados obtenidos.

Este objetivo recoge los tres ejes principales del proyecto. El primero es el estudio de modelos de detección de pose, entendidos como sistemas capaces de estimar automáticamente la localización de puntos corporales en imágenes o vídeos. El segundo es la evaluación técnica de los resultados generados, prestando atención a aspectos como la precisión visual, la continuidad

temporal, la confianza de los puntos detectados y la presencia de errores o anomalías en el esqueleto estimado. El tercero es la construcción de una herramienta software que permita aplicar estos modelos de forma práctica, ofreciendo una interfaz gráfica que facilite la interacción con el sistema y la interpretación de los resultados.

El proyecto toma como contexto de aplicación los vídeos relacionados con la ELA y otros entornos terapéuticos o neuromotores, ya que este tipo de secuencias pueden presentar movimientos lentos, limitaciones funcionales, cambios posturales, oclusiones parciales o dificultades adicionales para los modelos de visión artificial. No obstante, el objetivo del trabajo se mantiene dentro del ámbito informático: evaluar el comportamiento de los modelos y no emitir conclusiones clínicas sobre el estado de una persona.

2.2. Objetivos específicos

Para alcanzar el objetivo general se plantean una serie de objetivos específicos que estructuran el desarrollo del proyecto y permiten abordar cada una de sus partes de forma ordenada.

Estudiar los fundamentos de la detección de pose humana

El primer objetivo específico consiste en comprender los fundamentos de la detección de pose humana dentro del campo de la visión por computador. Para ello, será necesario estudiar qué se entiende por estimación de pose, qué tipos de aproximaciones existen, cómo se representan los puntos corporales y qué diferencias hay entre modelos de pose 2D, 3D o híbridos.

Este objetivo incluye el análisis de conceptos como keypoints, esqueletos, mapas de calor, coordenadas normalizadas, confianza de detección, modelos top-down y bottom-up, seguimiento temporal y representación estructurada del cuerpo humano. Comprender estos conceptos resulta imprescindible para poder comparar modelos de forma razonada y para diseñar posteriormente una aplicación que gestione correctamente sus salidas.

Además, este estudio permitirá establecer una terminología coherente para toda la memoria. Dado que distintos modelos utilizan formatos diferentes, será necesario definir con claridad qué se considera un punto corporal, cómo se interpreta su confianza, cómo se organizan los datos por frame y qué criterios se utilizarán para evaluar la calidad del esqueleto generado.

Revisar modelos de detección de pose basados en aprendizaje profundo

Otro objetivo fundamental es realizar una revisión de los principales modelos de detección de pose humana que puedan ser relevantes para el proyecto. Entre los modelos previstos se encuentran MediaPipe Pose, MoveNet, YOLO-Pose, OpenPose, AlphaPose, HRNet, RTMPose y ViTPose, aunque la selección final podrá adaptarse en función de la disponibilidad, documentación, facilidad de integración y adecuación al contexto del trabajo.

La revisión de estos modelos no se limitará a una descripción superficial, sino que deberá considerar aspectos técnicos como el tipo de arquitectura, el número de puntos corporales detectados, el formato de salida, la velocidad de inferencia, la precisión reportada, los requisitos hardware, la facilidad de instalación, la licencia de uso y la compatibilidad con aplicaciones de escritorio. También será importante valorar si el modelo está pensado para una sola persona o para múltiples personas, si ofrece información 2D o 3D, y si dispone de implementaciones estables que puedan utilizarse en el desarrollo del programa.

Este objetivo permitirá seleccionar de forma justificada los modelos más adecuados para una posible integración en la aplicación. No todos los modelos estudiados tendrán necesariamente que implementarse, ya que algunos pueden resultar demasiado pesados, complejos o poco prácticos para el alcance del TFG. Aun así, su análisis será útil para contextualizar el estado actual de la estimación de pose humana y para justificar las decisiones adoptadas.

Buscar y analizar bases de datos adecuadas

El proyecto requiere trabajar con vídeos ya existentes que permitan probar y evaluar los modelos de detección de pose. Por ello, otro objetivo específico consiste en localizar, estudiar y seleccionar bases de datos públicas o recursos audiovisuales que puedan resultar adecuados para el trabajo.

La búsqueda se orientará hacia distintos tipos de datos. Por un lado, se considerarán datasets generales de estimación de pose humana, útiles para comprender el formato habitual de anotación y la evaluación de modelos. Por otro lado, se buscarán datasets de ejercicios terapéuticos, rehabilitación, marcha humana, análisis de movimiento o actividades físicas controladas. También se prestará atención a recursos relacionados con enfermedades neuromotoras o neurodegenerativas, especialmente la ELA, siempre que existan datos disponibles y utilizables en el marco académico del proyecto.

El análisis de estas bases de datos deberá tener en cuenta aspectos como el tipo de vídeo, la resolución, la duración, el número de sujetos, la existencia o no de anotaciones, las condiciones de grabación, las restricciones de uso y la adecuación a los modelos de detección de pose seleccionados. La disponibilidad de buenos datos condiciona en gran medida la calidad de las pruebas, por lo que esta fase será esencial para garantizar que la evaluación realizada tenga sentido.

Definir una representación común para los esqueletos generados

Dado que cada modelo puede devolver un número diferente de puntos corporales y utilizar formatos distintos, uno de los objetivos técnicos más importantes será definir una representación común de los esqueletos generados. Esta representación permitirá almacenar y procesar las salidas de los modelos de forma homogénea, independientemente de la herramienta utilizada para obtenerlas.

La representación común deberá incluir información como el número de frame, el instante temporal del vídeo, el nombre del modelo utilizado, el identificador de la persona detectada, el nombre del punto corporal, sus coordenadas en la imagen, una posible coordenada de profundidad si está disponible y el valor de confianza asociado a la detección. Este formato facilitará la comparación entre modelos, el cálculo de métricas y la exportación posterior de resultados.

Este objetivo es especialmente relevante desde el punto de vista software, ya que permite separar la lógica de la aplicación de los detalles concretos de cada modelo. De esta forma, el programa podrá trabajar con una estructura interna unificada, aunque cada modelo tenga una API distinta o genere salidas en formatos diferentes. Esta decisión mejorará la modularidad, mantenibilidad y escalabilidad del sistema.

Establecer métricas de evaluación de la calidad de detección

Otro objetivo esencial consiste en definir métricas que permitan evaluar la calidad de los esqueletos generados por los modelos de detección de pose. Estas métricas deberán estar orientadas a medir el comportamiento técnico del modelo y no a valorar clínicamente al sujeto que aparece en el vídeo.

Entre las métricas previstas se encuentran la confianza media de los keypoints, el porcentaje de frames con detección completa, el número de puntos perdidos por secuencia, la estabilidad temporal de las coordenadas, la suavidad de las trayectorias, la variación de distancias entre articulaciones, la coherencia espacial del esqueleto y el tiempo de procesamiento por frame. También se podrá analizar la velocidad media de inferencia, expresada en frames por segundo, con el fin de valorar la viabilidad de cada modelo en una aplicación interactiva.

Estas métricas permitirán comparar los modelos bajo criterios objetivos y repetibles. Aunque en algunos casos no se disponga de anotaciones manuales que funcionen como verdad de referencia, será posible evaluar aspectos funcionales del comportamiento del modelo, como su estabilidad, completitud y consistencia. De este modo, el análisis no dependerá únicamente de una inspección visual subjetiva, sino que incorporará medidas cuantitativas.

Detectar anomalías en las salidas de los modelos de pose

El proyecto también tiene como objetivo diseñar mecanismos para detectar anomalías en los esqueletos generados por los modelos. Como se ha indicado anteriormente, estas anomalías se entienden como errores o comportamientos inesperados de la detección de pose, no como alteraciones médicas del movimiento humano.

Para ello, se estudiarán diferentes tipos de inconsistencias. Por ejemplo, se podrán identificar saltos bruscos de un keypoint entre frames consecutivos, pérdidas temporales de articulaciones, cambios repentinos en la confianza de detección, posiciones anatómicamente improbables, inversiones entre puntos izquierdos y derechos o variaciones excesivas en la longitud relativa de segmentos corporales. Estas situaciones pueden indicar que el modelo ha perdido precisión en un momento determinado o que las condiciones del vídeo dificultan una estimación fiable.

La detección de estas anomalías permitirá enriquecer la comparación entre modelos, ya que no solo se evaluará si un modelo detecta una pose, sino también cómo de estable y coherente es dicha detección a lo largo del tiempo. Este objetivo aportará una dimensión adicional al trabajo y reforzará su carácter informático, al centrarse en el análisis de calidad de las salidas generadas por algoritmos de visión artificial.

Diseñar una aplicación de escritorio con interfaz gráfica

Además del análisis teórico y experimental, el proyecto contempla el diseño de una aplicación de escritorio que permita utilizar de forma práctica los modelos seleccionados. El objetivo es que esta aplicación no sea únicamente una prueba de concepto básica, sino una herramienta con una estructura clara, una interfaz comprensible y una apariencia suficientemente sólida para ser presentada como resultado del TFG.

La aplicación deberá permitir cargar vídeos, seleccionar el modelo de detección de pose, ejecutar el procesamiento, visualizar el esqueleto superpuesto sobre el vídeo, mostrar métricas de calidad y señalar posibles anomalías detectadas en la salida del modelo. También deberá ofrecer opciones para exportar los resultados en formatos reutilizables, como CSV o JSON, e idealmente generar gráficas o informes que faciliten el análisis posterior.

El diseño de la interfaz gráfica deberá tener en cuenta la claridad, la usabilidad y la organización de la información. El usuario debe poder comprender fácilmente qué vídeo se está procesando, qué modelo se está utilizando, qué resultados se han obtenido y en qué puntos de la secuencia se han detectado posibles inconsistencias. Esta parte del proyecto permitirá demostrar la aplicación práctica de los conocimientos adquiridos y conectar el análisis de modelos con una herramienta software funcional.

Comparar los modelos seleccionados en condiciones homogéneas

Para que la comparación entre modelos sea válida, otro objetivo específico será definir un protocolo de evaluación homogéneo. Todos los modelos deberán aplicarse, en la medida de lo posible, sobre los mismos vídeos o secuencias, utilizando criterios similares de procesamiento y recogiendo las mismas métricas de salida.

Este objetivo implica establecer una metodología clara para las pruebas. Será necesario decidir qué vídeos se utilizan, qué fragmentos se analizan, cómo se almacenan los resultados, qué métricas se calculan y cómo se presentan las comparaciones. También será importante documentar las condiciones de ejecución, como el equipo utilizado, la resolución de entrada, el uso de CPU o GPU y las versiones de las librerías empleadas.

La comparación homogénea permitirá extraer conclusiones más fiables sobre las ventajas y limitaciones de cada modelo. Por ejemplo, un modelo puede ser más rápido pero generar esqueletos menos estables, mientras que

otro puede ofrecer mayor precisión visual pero requerir más recursos computacionales. El objetivo no será declarar un único modelo como universalmente superior, sino analizar cuál resulta más adecuado según los requisitos del sistema y el tipo de vídeo procesado.

Documentar el proceso de desarrollo y evaluación

Finalmente, uno de los objetivos del proyecto es documentar de forma completa el proceso seguido. La memoria deberá recoger no solo los resultados finales, sino también las decisiones tomadas, los criterios utilizados para seleccionar modelos y datasets, las dificultades encontradas y las limitaciones del sistema.

Esta documentación es importante porque permite que el trabajo sea comprensible, reproducible y evaluable. Un proyecto de este tipo no se mide únicamente por el programa final, sino también por la claridad con la que se justifican las decisiones técnicas, se explican los métodos utilizados y se interpretan los resultados obtenidos. Por ello, la memoria, los anexos y el código fuente deberán formar un conjunto coherente que refleje tanto el trabajo de investigación como el desarrollo software realizado.

2.3. Objetivos derivados de los requisitos del software

Además de los objetivos generales y específicos del proyecto, es necesario definir los objetivos asociados al software que se pretende construir. Estos objetivos describen las funcionalidades que deberá ofrecer la aplicación y las condiciones mínimas que deberá cumplir para resultar útil dentro del alcance del TFG.

Permitir la carga y gestión de vídeos

La aplicación deberá permitir al usuario cargar vídeos desde el sistema de archivos local. Estos vídeos podrán proceder de datasets públicos, recursos terapéuticos o grabaciones de prueba utilizadas durante el desarrollo. El sistema deberá ser capaz de leer el archivo, obtener información básica como duración, resolución y número de frames, y prepararlo para su procesamiento mediante los modelos de detección de pose.

Este requisito es fundamental, ya que el vídeo constituye la entrada principal del sistema. La aplicación deberá gestionar correctamente distintos

formatos habituales, siempre dentro de las posibilidades de las librerías utilizadas. También será conveniente que el usuario pueda visualizar el vídeo original y, posteriormente, compararlo con el vídeo procesado o con la representación del esqueleto generada por el modelo.

Integrar varios modelos de detección de pose

Uno de los requisitos centrales del software será la posibilidad de trabajar con más de un modelo de detección de pose. La aplicación deberá diseñarse de forma modular para que cada modelo pueda integrarse como un componente independiente, manteniendo una interfaz común de entrada y salida.

Inicialmente, los modelos más adecuados para una posible integración serán aquellos que ofrezcan una buena relación entre facilidad de uso, documentación, rendimiento y estabilidad. MediaPipe Pose, MoveNet y YOLO-Pose se perfilan como candidatos especialmente interesantes. No obstante, el diseño deberá permitir que en el futuro puedan añadirse otros modelos sin modificar de forma profunda la arquitectura del sistema.

Visualizar los esqueletos generados

El software deberá mostrar de forma gráfica los esqueletos obtenidos por el modelo seleccionado. Esta visualización podrá realizarse superponiendo los puntos y conexiones del esqueleto sobre el vídeo original, de forma que el usuario pueda observar directamente cómo el modelo interpreta la pose de la persona en cada frame.

La visualización será importante tanto para la interpretación de resultados como para la detección manual de posibles errores. Aunque el sistema calcule métricas automáticas, la revisión visual seguirá siendo útil para comprobar si el comportamiento del modelo es coherente. Por ello, la interfaz deberá ofrecer una representación clara, evitando que la información gráfica dificulte la lectura del vídeo.

Calcular y mostrar métricas de calidad

La aplicación deberá calcular métricas relacionadas con la calidad de la detección de pose. Estas métricas podrán mostrarse al usuario de forma resumida, mediante tablas, indicadores o gráficas. El objetivo es que el usuario pueda comparar modelos no solo visualmente, sino también a partir de datos cuantitativos.

Entre las métricas que podrían mostrarse se incluyen la confianza media del modelo, el número de frames procesados correctamente, el porcentaje de puntos detectados, la cantidad de anomalías encontradas, la estabilidad temporal media y el tiempo de procesamiento. Estas medidas permitirán valorar de forma más objetiva el rendimiento de cada modelo.

Detectar y señalar inconsistencias en la detección

El sistema deberá incorporar mecanismos para identificar posibles anomalías en los esqueletos generados. Cuando se detecte una inconsistencia, la aplicación debería poder señalar el frame afectado, el keypoint implicado y el tipo de problema observado, siempre que esta información pueda calcularse de forma razonable.

Este requisito puede aportar mucho valor al programa, ya que permitirá localizar automáticamente momentos del vídeo en los que el modelo no se comporta de forma fiable. De este modo, el usuario no tendrá que revisar manualmente toda la secuencia para encontrar errores evidentes. Además, esta información podrá utilizarse posteriormente para comparar modelos y analizar su robustez.

Exportar resultados

La aplicación deberá permitir la exportación de resultados para su análisis externo o inclusión en los anexos del TFG. Los resultados podrán guardarse en formatos como CSV, JSON o similares, incluyendo información sobre los frames procesados, las coordenadas de los keypoints, los valores de confianza, las métricas calculadas y las anomalías detectadas.

La exportación es importante porque permite conservar los resultados, compararlos posteriormente y generar tablas o gráficas para la memoria. Además, facilita la reproducibilidad del trabajo, ya que los datos obtenidos por la aplicación pueden revisarse sin necesidad de volver a procesar todos los vídeos.

Ofrecer una interfaz clara y usable

La interfaz gráfica deberá ser sencilla, ordenada y comprensible. Aunque el proyecto tenga una base técnica compleja, el usuario no debería necesitar conocimientos profundos sobre modelos de pose para utilizar las funciones principales de la aplicación.

El diseño deberá separar claramente las zonas de carga de vídeo, selección de modelo, visualización, métricas y exportación. También será conveniente incluir mensajes de estado, barras de progreso o avisos que informen al usuario del avance del procesamiento. Una interfaz bien diseñada contribuirá a que el programa tenga una apariencia más profesional y a que el resultado final sea más sólido.

2.4. Objetivos técnicos del proyecto

Junto a los objetivos derivados de los requisitos del software, el proyecto plantea una serie de retos técnicos relacionados con la implementación, la integración de modelos y el procesamiento de resultados.

Diseñar una arquitectura modular

Uno de los objetivos técnicos más importantes será diseñar una arquitectura software modular. Esto implica dividir el programa en componentes independientes, cada uno encargado de una función concreta: gestión de vídeo, integración de modelos, normalización de keypoints, cálculo de métricas, detección de anomalías, visualización y exportación.

Una arquitectura modular facilitará el desarrollo progresivo del sistema y permitirá sustituir o añadir modelos sin alterar el resto de la aplicación. También hará que el código sea más fácil de mantener, depurar y documentar. Este aspecto será especialmente importante si se pretende que el programa pueda evolucionar en el futuro.

Normalizar las salidas de modelos diferentes

Cada modelo de detección de pose puede generar salidas distintas. Algunos detectan 17 puntos, otros 25, 33 o incluso más. Algunos ofrecen únicamente coordenadas 2D, mientras que otros añaden una estimación de profundidad o información adicional. Además, los nombres de los puntos y el orden en que aparecen pueden variar entre modelos.

Por ello, uno de los retos técnicos será transformar estas salidas en una estructura común que pueda ser utilizada por el resto de módulos de la aplicación. Esta normalización permitirá comparar modelos, calcular métricas de forma homogénea y exportar resultados con un formato coherente.

Gestionar la eficiencia del procesamiento de vídeo

El procesamiento de vídeo puede ser costoso, especialmente si se aplican modelos de aprendizaje profundo frame a frame. Por tanto, será necesario prestar atención a la eficiencia del sistema. El objetivo será lograr un equilibrio entre calidad de análisis y tiempo de procesamiento.

Para ello, podrán estudiarse estrategias como redimensionar frames, procesar solo determinados fragmentos del vídeo, permitir la selección de resolución, utilizar modelos ligeros cuando sea necesario o almacenar resultados intermedios para evitar repetir cálculos. La eficiencia será importante para que la aplicación sea utilizable en un entorno realista.

Evaluar la estabilidad temporal de los keypoints

Uno de los aspectos técnicos más relevantes será analizar la estabilidad temporal de los puntos detectados. En una secuencia de vídeo, un keypoint debería moverse de forma relativamente continua, salvo que exista un cambio brusco real en el movimiento. Si un punto salta de una posición a otra sin coherencia, desaparece repentinamente o cambia de lado, puede tratarse de un error del modelo.

Evaluar esta estabilidad requerirá comparar la posición de los puntos entre frames consecutivos, calcular variaciones, detectar cambios abruptos y establecer criterios para decidir cuándo una desviación puede considerarse anómala. Este objetivo conecta directamente con la detección de anomalías técnicas en los esqueletos generados.

Mantener la trazabilidad de los resultados

El sistema deberá permitir relacionar cada resultado con el vídeo, modelo, frame y configuración utilizados. Esta trazabilidad será importante para interpretar correctamente las métricas y para poder reproducir los experimentos realizados.

Cada salida generada deberá incluir información suficiente para saber de dónde procede. Por ejemplo, no bastará con guardar una coordenada aislada, sino que deberá saberse a qué frame pertenece, qué modelo la generó, cuál era el punto corporal correspondiente y qué nivel de confianza tenía. Esta organización facilitará el análisis posterior y la elaboración de la memoria.

Preparar el sistema para futuras ampliaciones

Aunque el TFG tendrá un alcance limitado, el diseño del sistema deberá contemplar posibles ampliaciones futuras. Entre ellas podrían incluirse la integración de nuevos modelos, la incorporación de técnicas de filtrado temporal, la comparación con anotaciones manuales, la generación automática de informes, el uso de modelos 3D o la adaptación a otros tipos de vídeos.

Este objetivo no implica implementar todas esas funcionalidades durante el proyecto, sino diseñar el software de manera que no cierre la puerta a futuras mejoras. Un sistema bien planteado debe poder crecer sin requerir una reescritura completa.

2.5. Alcance del proyecto

El alcance del proyecto queda definido por los objetivos anteriores. El trabajo se centrará en el análisis de modelos de detección de pose humana aplicados a vídeos 2D, la evaluación de la calidad de los esqueletos generados y el diseño de una aplicación de escritorio para visualizar y comparar resultados.

Quedan fuera del alcance del proyecto el diagnóstico médico automático, la predicción de evolución clínica de pacientes, la sustitución de valoraciones realizadas por profesionales sanitarios y la validación clínica del sistema en entornos hospitalarios. Aunque la ELA se utiliza como contexto de aplicación y motivación, el proyecto se desarrolla desde una perspectiva informática, centrada en el comportamiento de los modelos de visión por computador.

También se debe tener en cuenta que la comparación entre modelos estará condicionada por la disponibilidad de datasets adecuados, las limitaciones de hardware y la documentación existente de cada herramienta. Por ello, el proyecto buscará un equilibrio entre profundidad técnica y viabilidad práctica, priorizando aquellos modelos que puedan estudiarse, probarse y, en su caso, integrarse de forma razonable en una aplicación funcional.

3. Conceptos teóricos

En este capítulo se presentan los conceptos teóricos necesarios para comprender el desarrollo del proyecto. El trabajo se sitúa dentro del ámbito de la visión por computador, el aprendizaje profundo y la detección de pose humana en vídeos 2D. Aunque el contexto de aplicación se relaciona con vídeos médicos, terapéuticos o de rehabilitación vinculados a la esclerosis lateral amiotrófica, el enfoque principal del proyecto es informático. Por tanto, el objetivo no es realizar una valoración clínica, sino analizar la calidad, estabilidad y coherencia de los esqueletos generados por distintos modelos de detección de pose.

3.1. Esclerosis lateral amiotrófica

La esclerosis lateral amiotrófica, conocida como ELA, es una enfermedad neurodegenerativa progresiva que afecta a las neuronas motoras encargadas del control de la musculatura voluntaria. Como consecuencia, puede producir pérdida de fuerza, dificultades de movilidad, alteraciones en la coordinación y limitaciones funcionales cada vez mayores [6, 12].

En este proyecto, la ELA se utiliza como contexto de aplicación porque los vídeos relacionados con alteraciones neuromotoras pueden presentar movimientos lentos, posturas poco habituales, apoyos externos u oclusiones parciales. Estas condiciones pueden dificultar la estimación de pose y resultan interesantes para evaluar la robustez de los modelos.

No obstante, el trabajo no pretende diagnosticar la enfermedad ni valorar clínicamente a los pacientes. La finalidad es estudiar, desde un punto de vista técnico, cómo se comportan los modelos de detección de pose al procesar

vídeos 2D y hasta qué punto los esqueletos generados son fiables para un análisis posterior.

3.2. Visión por computador y aprendizaje profundo

La visión por computador es una disciplina de la inteligencia artificial cuyo objetivo es permitir que un sistema informático extraiga información útil a partir de imágenes o vídeos. Entre sus tareas más habituales se encuentran la clasificación de imágenes, la detección de objetos, la segmentación, el seguimiento y la estimación de pose humana [16].

En el caso del vídeo, el sistema debe analizar una secuencia de frames ordenados temporalmente. Esto resulta especialmente importante en el análisis de movimiento, ya que los puntos corporales estimados deberían evolucionar de forma coherente entre frames consecutivos. Un salto brusco o una pérdida repentina de un punto puede indicar una inconsistencia del modelo.

Flujo de procesamiento de video para detección de pose

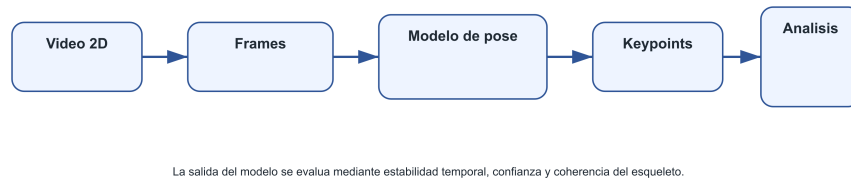


Figura 3.1: Flujo general de procesamiento de vídeo mediante técnicas de visión por computador. Fuente: elaboración propia.

El aprendizaje profundo, o *deep learning*, se basa en modelos formados por múltiples capas capaces de aprender representaciones complejas a partir de los datos [4]. En visión por computador, estos modelos han permitido mejorar notablemente el rendimiento en tareas visuales, especialmente desde la aparición de arquitecturas convolucionales profundas aplicadas a grandes conjuntos de imágenes [9].

3.3. Redes neuronales convolucionales

Las redes neuronales convolucionales, conocidas como CNN, son un tipo de red neuronal especialmente diseñado para trabajar con imágenes. Su principal característica es el uso de operaciones de convolución, que aplican filtros sobre pequeñas regiones de la imagen para extraer patrones locales como bordes, texturas o formas [10].

Una CNN suele estar formada por capas convolucionales, funciones de activación, capas de reducción o *pooling* y capas finales encargadas de generar la salida del modelo. Gracias a esta estructura, las CNN pueden aprender características visuales de forma jerárquica: las primeras capas detectan patrones simples, mientras que las capas más profundas representan estructuras más complejas.

Esquema simplificado de una red neuronal convolucional



Las primeras capas extraen patrones simples y las capas posteriores combinan esas características en representaciones más complejas.

Figura 3.2: Esquema simplificado de una red neuronal convolucional aplicada al análisis de imágenes. Fuente: elaboración propia.

En detección de pose humana, las CNN han sido fundamentales para localizar partes del cuerpo en imágenes. Aunque algunos modelos actuales incorporan arquitecturas más recientes, como transformadores o mecanismos de atención, muchas soluciones siguen utilizando redes convolucionales o componentes derivados de ellas para extraer características visuales.

3.4. Detección de pose humana

La detección de pose humana consiste en estimar la posición de puntos anatómicos relevantes del cuerpo en una imagen o vídeo. Estos puntos,

denominados *keypoints*, suelen corresponder a articulaciones o zonas corporales como hombros, codos, muñecas, caderas, rodillas, tobillos, nariz u ojos [18, 2].

A partir de estos *keypoints* se puede construir una representación simplificada del cuerpo humano mediante un esqueleto. Esta representación permite analizar la postura y el movimiento sin trabajar directamente con todos los píxeles del vídeo. En este proyecto, dicha representación se utiliza como salida técnica de los modelos de detección de pose.

Los métodos de detección de pose suelen clasificarse en dos enfoques principales. En los métodos *top-down*, primero se detecta a la persona y después se estiman sus puntos corporales. En los métodos *bottom-up*, primero se detectan los puntos corporales presentes en la imagen y posteriormente se agrupan para formar esqueletos completos [2].

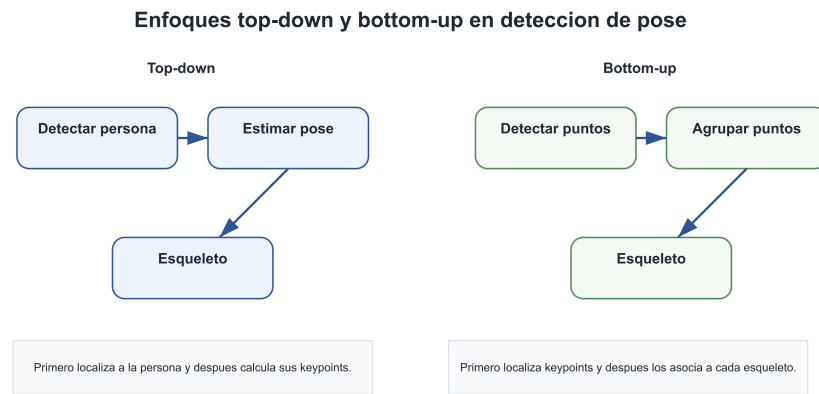


Figura 3.3: Comparación conceptual entre enfoques *top-down* y *bottom-up* en detección de pose humana. Fuente: elaboración propia.

3.5. Modelos actuales de detección de pose

Existen numerosos modelos de detección de pose, cada uno con características distintas en cuanto a precisión, velocidad, número de puntos detectados, facilidad de integración y requisitos computacionales. Esta variedad hace necesario estudiar varios modelos antes de decidir cuáles resultan más adecuados para una aplicación de escritorio.

Entre los modelos más relevantes se encuentra OpenPose, uno de los sistemas más conocidos en estimación de pose multipersona. Su enfoque se

basa en la detección de puntos corporales y su asociación mediante campos de afinidad de partes, lo que lo convierte en una referencia importante dentro de los métodos *bottom-up* [2]. Sin embargo, su instalación y coste computacional pueden hacerlo menos práctico para una primera implementación.

MediaPipe Pose es una alternativa especialmente interesante por su facilidad de integración y su rendimiento en tiempo real. Proporciona 33 landmarks corporales, lo que ofrece una representación más detallada que otros modelos basados en 17 puntos [5]. MoveNet, por su parte, está orientado a una detección rápida y precisa de 17 keypoints, con variantes que permiten priorizar velocidad o precisión [17].

YOLO-Pose también resulta relevante por su enfoque práctico y eficiente. Las herramientas basadas en YOLO permiten combinar detección de personas y estimación de keypoints en modelos rápidos y relativamente sencillos de utilizar [19]. Otros modelos como AlphaPose, HRNet, RTMPose o ViTPose son importantes para el estado del arte, aunque pueden requerir una integración más compleja [3, 15, 8, 20].

Para este proyecto, los modelos más adecuados para una primera integración serían MediaPipe Pose, MoveNet y YOLO-Pose, mientras que el resto pueden estudiarse como referencias comparativas o posibles ampliaciones futuras.

3.6. Representación de la pose mediante keypoints

La salida de un modelo de detección de pose suele representarse mediante un conjunto de keypoints. Cada keypoint incluye normalmente unas coordenadas en la imagen y un valor de confianza asociado. Este valor indica el grado de seguridad que el modelo asigna a la posición estimada.

El número de puntos corporales depende del modelo utilizado. El formato COCO, por ejemplo, define 17 keypoints principales del cuerpo humano [11]. En cambio, MediaPipe Pose utiliza 33 landmarks, incluyendo puntos adicionales en manos, pies y rostro [5]. Esta diferencia hace necesario definir una representación común si se quieren comparar varios modelos dentro de una misma aplicación.

Una representación común podría incluir el identificador del frame, el instante temporal, el modelo utilizado, el nombre del keypoint, sus coordena-

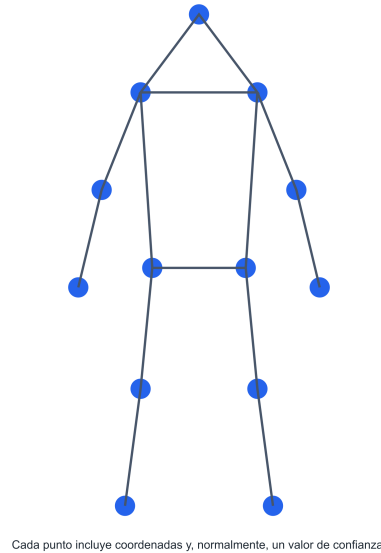
Representacion de pose mediante keypoints

Figura 3.4: Representación esquemática de una pose humana mediante puntos corporales y conexiones entre articulaciones. Fuente: elaboración propia.

das y su confianza. Además, conviene normalizar las coordenadas respecto al tamaño de la imagen para poder comparar vídeos con resoluciones diferentes.

3.7. Evaluación técnica de modelos de detección de pose

La evaluación de modelos de detección de pose puede realizarse mediante métricas clásicas cuando se dispone de datasets anotados. Entre ellas se encuentran AP, mAP, PCK u OKS, que comparan los puntos estimados por el modelo con anotaciones de referencia [11].

Sin embargo, en vídeos terapéuticos o procedentes de bases de datos reales no siempre existen anotaciones manuales. Por ello, este proyecto también considera métricas funcionales relacionadas con la calidad de la detección, como la confianza media de los keypoints, el porcentaje de frames con pose válida, la estabilidad temporal de los puntos, la coherencia de las distancias entre articulaciones y la velocidad de procesamiento.

Estos criterios permiten comparar modelos desde un punto de vista práctico. Un modelo puede ser muy rápido pero generar esqueletos inestables, mientras que otro puede ser más preciso pero requerir mayor coste computacional. Por tanto, la comparación debe valorar tanto la calidad de la salida como la viabilidad de integrar el modelo en una aplicación de escritorio.

3.8. Anomalías en la estimación de esqueletos

En este proyecto, una anomalía en la estimación de esqueletos no hace referencia a una alteración médica del sujeto, sino a una inconsistencia técnica generada por el modelo de detección de pose. El objetivo es identificar errores o comportamientos poco fiables en la salida del modelo.

Algunos ejemplos de anomalías técnicas son el desplazamiento brusco de un keypoint entre frames consecutivos, la pérdida temporal de una articulación, una confianza anormalmente baja, la inversión entre puntos izquierdos y derechos o una deformación artificial del esqueleto. Estas situaciones pueden aparecer por oclusiones, baja calidad del vídeo, posturas poco habituales o limitaciones del propio modelo.

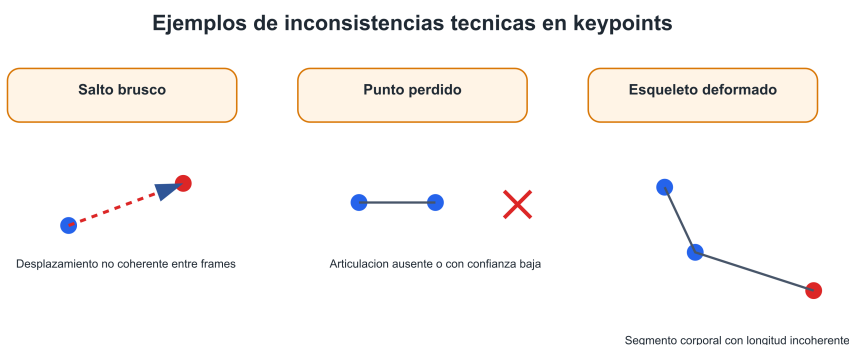


Figura 3.5: Ejemplos de posibles inconsistencias técnicas en la estimación de keypoints entre frames consecutivos. Fuente: elaboración propia.

La detección de estas anomalías puede realizarse mediante reglas sencillas, como umbrales de desplazamiento, umbrales de confianza o análisis de la

variación de distancias entre articulaciones. Este enfoque es adecuado para el alcance del proyecto porque permite obtener resultados interpretables y directamente relacionados con la calidad técnica del modelo.

3.9. Aplicación del análisis de pose en vídeos terapéuticos

Los vídeos terapéuticos constituyen un escenario interesante para evaluar modelos de detección de pose, ya que suelen presentar condiciones más complejas que otros vídeos generales. En ellos pueden aparecer movimientos lentos, posturas sentadas, apoyos externos, asistencia de terapeutas, oclusiones parciales o ángulos de cámara poco ideales.

Desde el punto de vista informático, estas condiciones permiten comprobar la robustez de los modelos. Un sistema que funciona correctamente en imágenes claras y controladas puede generar errores cuando trabaja con vídeos reales o semirrealistas. Por ello, aplicar los modelos sobre este tipo de secuencias permite analizar su estabilidad, precisión y capacidad para mantener esqueletos coherentes a lo largo del tiempo.

La aplicación final del proyecto aprovechará este análisis para cargar vídeos, aplicar distintos modelos de detección de pose, visualizar los esqueletos generados, calcular métricas de calidad y señalar posibles inconsistencias. De este modo, el sistema se plantea como una herramienta técnica de comparación y evaluación de modelos, no como una solución clínica de diagnóstico o valoración médica.

4. Técnicas y herramientas

En este capítulo se describen las principales técnicas metodológicas y herramientas de desarrollo consideradas para la realización del proyecto. El objetivo no es profundizar de forma exhaustiva en cada tecnología, sino justificar su papel dentro del Trabajo de Fin de Grado y explicar por qué determinadas opciones resultan más adecuadas que otras para el sistema planteado.

El proyecto combina varias áreas técnicas: procesamiento de vídeo, detección de pose humana, análisis de datos, detección de inconsistencias en las salidas de los modelos y desarrollo de una aplicación de escritorio. Por ello, la elección de herramientas debe responder tanto a criterios de rendimiento como a criterios de viabilidad, facilidad de integración, documentación disponible y compatibilidad entre bibliotecas.

Desde el inicio se ha priorizado una solución que permita trabajar con modelos de aprendizaje profundo ya existentes, procesar vídeos 2D frame a frame, extraer coordenadas de puntos corporales, analizar la calidad de los esqueletos generados y presentar los resultados mediante una interfaz gráfica. Además, se ha buscado que el entorno de desarrollo sea accesible, reproducible y adecuado para un proyecto académico.

4.1. Metodología de desarrollo

Para el desarrollo del proyecto se plantea una metodología incremental basada en prototipos. Esta decisión responde a la naturaleza experimental del trabajo, ya que no se trata únicamente de construir una aplicación cerrada desde el primer momento, sino de estudiar modelos de detección

de pose, probar herramientas, comparar resultados y adaptar el diseño del sistema en función de las conclusiones obtenidas.

En una primera fase, el trabajo se centra en el análisis teórico y la selección de modelos. Esta etapa permite estudiar las características de diferentes alternativas, como MediaPipe Pose, MoveNet, YOLO-Pose, OpenPose, AlphaPose, HRNet, RTMPose o ViTPose. A partir de esta comparación, se pueden identificar los modelos más adecuados para una futura integración en la aplicación, teniendo en cuenta aspectos como facilidad de instalación, velocidad, precisión, número de keypoints y formato de salida.

En una segunda fase, se plantea el desarrollo de prototipos independientes. Estos prototipos servirán para comprobar que cada herramienta funciona correctamente antes de integrarla en una aplicación completa. Por ejemplo, se podrá crear un primer prototipo para cargar un vídeo y extraer frames, otro para aplicar un modelo de detección de pose sobre imágenes individuales, y otro para almacenar keypoints en un formato estructurado. Esta separación reduce el riesgo técnico, ya que permite validar cada componente de forma aislada.

Posteriormente, los prototipos se integrarán progresivamente en una aplicación de escritorio. Este enfoque incremental es adecuado porque permite construir el sistema por módulos: gestión de vídeo, selección de modelo, visualización de resultados, normalización de keypoints, cálculo de métricas y detección de anomalías técnicas. Además, facilita la depuración y permite realizar mejoras sin tener que rediseñar todo el programa.

La metodología también contempla una fase de evaluación, donde se aplicarán los modelos seleccionados sobre los mismos vídeos o fragmentos de vídeo. El objetivo será comparar sus salidas bajo condiciones similares, analizando métricas como velocidad de procesamiento, confianza media, estabilidad temporal, pérdida de puntos y número de inconsistencias detectadas. De esta forma, la evaluación estará alineada con el enfoque informático del proyecto.

4.2. Lenguaje de programación

El lenguaje elegido para el desarrollo principal del proyecto es Python. Esta decisión se debe a que Python es uno de los lenguajes más utilizados en inteligencia artificial, aprendizaje automático, visión por computador y análisis de datos. Su sintaxis clara y su amplio ecosistema de bibliotecas

permiten desarrollar prototipos de forma rápida y conectar herramientas muy diferentes dentro de un mismo entorno [14].

Una de las principales ventajas de Python en este proyecto es la disponibilidad de bibliotecas especializadas. Herramientas como OpenCV, TensorFlow, MediaPipe, NumPy, Pandas o Matplotlib cuentan con soporte para Python, lo que facilita el procesamiento de vídeo, la ejecución de modelos de detección de pose, el análisis numérico y la generación de resultados. Esta compatibilidad reduce la complejidad del desarrollo y permite centrarse en la lógica del sistema.

También se valoraron otras alternativas, como C++ o C#. C++ puede ofrecer un rendimiento elevado y es utilizado internamente por muchas bibliotecas de visión por computador, pero su curva de desarrollo es más exigente. C#, por otro lado, permite crear interfaces gráficas muy sólidas en Windows mediante tecnologías como WPF o WinUI. Sin embargo, la integración con modelos de aprendizaje profundo y bibliotecas de visión artificial suele ser más directa en Python. Por esta razón, Python representa un equilibrio adecuado entre potencia, facilidad de uso y disponibilidad de herramientas.

En el contexto de este TFG, el rendimiento bruto no es el único criterio importante. Aunque el procesamiento de vídeo puede ser costoso, el objetivo principal es construir una aplicación funcional, modular y justificable académicamente. Python permite avanzar de forma más ágil y facilita la experimentación con distintos modelos, lo que resulta especialmente valioso en un proyecto de comparación técnica.

4.3. Entorno de desarrollo

Como entorno de desarrollo se propone el uso de Visual Studio Code. Se trata de un editor de código ampliamente utilizado, con soporte para múltiples lenguajes, extensiones, depuración integrada y una buena integración con entornos Python [13]. Su ligereza y flexibilidad lo convierten en una opción adecuada para proyectos donde se combinan scripts, módulos, archivos de configuración, documentación y pruebas.

Visual Studio Code permite trabajar con entornos virtuales de Python, gestionar dependencias, ejecutar código desde la terminal integrada y utilizar extensiones específicas para autocompletado, análisis estático y depuración. Estas características son especialmente útiles en un proyecto como este,

donde se prevé trabajar con varias bibliotecas externas y diferentes módulos de código.

Se consideró también el uso de Visual Studio clásico, especialmente por su capacidad para desarrollar aplicaciones de escritorio en C#. No obstante, dado que la mayor parte de las bibliotecas de inteligencia artificial y visión por computador se integran de forma más sencilla en Python, se considera más conveniente utilizar Visual Studio Code junto con Python como entorno principal de desarrollo. Esta combinación permite mantener un flujo de trabajo más directo y evita añadir una capa adicional de comunicación entre una interfaz en C# y un motor de procesamiento en Python.

Otra alternativa posible sería utilizar entornos como PyCharm, que también ofrece una experiencia sólida para proyectos Python. Sin embargo, Visual Studio Code resulta más ligero, flexible y suficiente para el alcance del proyecto. Además, permite trabajar cómodamente con archivos LaTeX, Markdown, JSON, CSV y scripts de Python dentro del mismo espacio de trabajo.

4.4. Procesamiento de vídeo con OpenCV

OpenCV es una biblioteca ampliamente utilizada en visión por computador. Proporciona herramientas para leer imágenes y vídeos, acceder a frames, modificar resoluciones, convertir espacios de color, dibujar sobre imágenes y guardar resultados procesados [1]. En este proyecto, OpenCV resulta especialmente importante porque actúa como puente entre los vídeos de entrada y los modelos de detección de pose.

El procesamiento de vídeo se basa en la lectura secuencial de frames. Cada vídeo puede descomponerse en una sucesión de imágenes, y cada una de estas imágenes puede ser procesada por un modelo de estimación de pose. OpenCV permite acceder a esta información de manera sencilla, obteniendo datos como el número de frames, la resolución, la tasa de imágenes por segundo y la duración aproximada del vídeo.

Otra función relevante de OpenCV es la visualización de resultados. Una vez que un modelo detecta los keypoints de una persona, es posible dibujar puntos, líneas o esqueletos sobre el frame original. Esta capacidad resulta útil tanto para la aplicación final como para las fases de prueba, ya que permite comprobar visualmente si el modelo está generando una salida coherente.

OpenCV también facilita tareas de preprocesamiento. En función del modelo utilizado, puede ser necesario redimensionar la imagen, cambiar

el formato de color, recortar regiones de interés o normalizar los datos de entrada. Estas operaciones son habituales en visión por computador y pueden afectar al rendimiento y calidad de la detección. Por ello, disponer de una biblioteca robusta para manipular imágenes y vídeos es fundamental.

Aunque existen otras alternativas, como usar directamente herramientas de vídeo específicas o bibliotecas de alto nivel, OpenCV ofrece una combinación muy adecuada de rendimiento, documentación y compatibilidad con Python. Además, su uso está muy extendido en proyectos académicos e industriales, lo que facilita encontrar ejemplos, documentación y soluciones a problemas comunes.

4.5. Modelos de detección de pose considerados

Una parte central del proyecto consiste en estudiar y comparar modelos de detección de pose humana. Estos modelos reciben como entrada una imagen o frame de vídeo y devuelven la posición estimada de distintos puntos corporales. Aunque todos persiguen un objetivo similar, existen diferencias importantes en el número de puntos detectados, la arquitectura interna, la velocidad, la precisión y la facilidad de integración.

Entre las alternativas estudiadas se encuentran MediaPipe Pose, MoveNet, YOLO-Pose, OpenPose, AlphaPose, HRNet, RTMPose y ViTPose. No obstante, no todos los modelos tienen por qué integrarse necesariamente en la aplicación final. Algunos se consideran principalmente como referencias del estado del arte, mientras que otros resultan más viables para una implementación práctica.

MediaPipe Pose es una de las opciones más interesantes para el proyecto por su facilidad de integración y su capacidad para estimar un conjunto amplio de landmarks corporales. La documentación oficial de MediaPipe describe tareas de visión orientadas a estimación de pose, incluyendo la detección de puntos corporales y su uso en imágenes o vídeo [5]. Su principal ventaja es que permite obtener resultados rápidos y suficientemente detallados para una aplicación de escritorio.

MoveNet, disponible a través del ecosistema TensorFlow, es otro modelo relevante. Está orientado a una estimación rápida y precisa de la pose humana, trabajando con 17 keypoints corporales y ofreciendo variantes que priorizan velocidad o precisión [17]. Esta característica lo convierte en una

alternativa adecuada para comparar el equilibrio entre rendimiento y calidad de detección.

YOLO-Pose representa una opción especialmente práctica por su integración dentro del ecosistema YOLO. Las herramientas de Ultralytics permiten utilizar modelos orientados a pose de forma relativamente sencilla, combinando detección y estimación de puntos corporales [19]. Su principal interés para el proyecto es su velocidad y su facilidad de uso en aplicaciones reales.

OpenPose, AlphaPose, HRNet, RTMPose y ViTPose se consideran herramientas relevantes para comprender el estado actual de la detección de pose. OpenPose es una referencia clásica en estimación multipersona mediante campos de afinidad de partes [2]. AlphaPose ofrece una aproximación orientada a escenarios multipersona con alta precisión [3]. HRNet destaca por mantener representaciones de alta resolución durante el procesamiento [15]. RTMPose se orienta a estimación multipersona eficiente en tiempo real [8]. ViTPose representa el uso de transformadores visuales en estimación de pose humana [20].

Para la aplicación final, se priorizarán aquellos modelos que ofrezcan un equilibrio razonable entre facilidad de instalación, estabilidad, velocidad y calidad de salida. En este sentido, MediaPipe Pose, MoveNet y YOLO-Pose se perfilan como las alternativas más adecuadas para una primera implementación, mientras que el resto se utilizarán como referencia comparativa o posibles líneas futuras.

4.6. Interfaz gráfica de usuario

Uno de los objetivos del proyecto es desarrollar una aplicación con una interfaz gráfica que permita al usuario cargar vídeos, seleccionar modelos de detección de pose, visualizar resultados y exportar datos. Para ello, se han considerado distintas alternativas, como Tkinter, PyQt, PySide6, interfaces web locales con Streamlit o una aplicación de escritorio desarrollada en C#.

Tkinter tiene la ventaja de estar incluido en muchas instalaciones de Python y permite crear interfaces sencillas. Sin embargo, su aspecto visual y su capacidad para construir aplicaciones más complejas pueden resultar limitados. Streamlit, por otro lado, permite crear interfaces web de forma muy rápida, pero está más orientado a prototipos y paneles interactivos que a una aplicación de escritorio tradicional.

PySide6 se considera una de las opciones más adecuadas para este proyecto. PySide6 forma parte de Qt for Python y proporciona acceso al framework Qt 6 desde Python. Qt es una tecnología ampliamente utilizada para construir interfaces gráficas multiplataforma, con soporte para ventanas, botones, menús, paneles, tablas, barras de progreso y otros componentes habituales en aplicaciones de escritorio.

La elección de PySide6 se justifica por varios motivos. En primer lugar, permite mantener todo el desarrollo en Python, evitando dividir el proyecto entre un frontend en otro lenguaje y un backend de procesamiento en Python. En segundo lugar, ofrece una apariencia más profesional que soluciones muy básicas. En tercer lugar, facilita organizar la aplicación en ventanas, paneles y módulos visuales, lo que encaja con las necesidades del programa planteado.

PyQt6 sería una alternativa muy similar, ya que también proporciona acceso a Qt desde Python. No obstante, PySide6 es la opción oficial del proyecto Qt for Python, lo que la convierte en una elección apropiada para un proyecto académico. En cualquier caso, ambas opciones comparten muchos conceptos y permitirían construir una interfaz de escritorio sólida.

4.7. Análisis y almacenamiento de datos

Una vez que los modelos de detección de pose generan los keypoints, es necesario almacenar, procesar y analizar esa información. Para ello se plantea el uso de herramientas habituales del ecosistema Python, como NumPy, Pandas y los formatos CSV y JSON.

NumPy es una biblioteca fundamental para cálculo numérico en Python. Permite trabajar de forma eficiente con vectores, matrices y operaciones matemáticas, lo que resulta útil para manipular coordenadas de keypoints, calcular distancias, variaciones temporales o métricas de estabilidad [?]. En este proyecto, NumPy puede utilizarse para operaciones de bajo nivel sobre los datos numéricos generados por los modelos.

Pandas, por su parte, permite organizar datos en estructuras tabulares, especialmente mediante objetos tipo *DataFrame*. Esta herramienta resulta adecuada para almacenar resultados por frame, modelo, persona y keypoint, facilitando el filtrado, agrupación y exportación de datos [?]. Por ejemplo, puede utilizarse para generar tablas con coordenadas, valores de confianza, métricas de calidad o anomalías detectadas.

El formato CSV será útil para exportar resultados tabulares, ya que puede abrirse fácilmente con hojas de cálculo y herramientas de análisis. El

formato JSON, en cambio, resulta más adecuado cuando se desea conservar una estructura jerárquica, por ejemplo agrupando resultados por vídeo, modelo, frame y persona. Python incluye soporte estándar para trabajar con ambos formatos mediante sus módulos de biblioteca estándar [?, ?].

El uso combinado de NumPy, Pandas, CSV y JSON permite que los resultados del programa sean reutilizables y fáciles de revisar. Esto es importante para la reproducibilidad del proyecto, ya que las salidas generadas por la aplicación podrán incluirse en anexos, compararse posteriormente o emplearse para generar gráficas.

4.8. Visualización de resultados

La visualización de resultados es importante tanto durante el desarrollo como en la aplicación final. Por un lado, es necesario mostrar el esqueleto generado sobre el vídeo para comprobar visualmente el comportamiento del modelo. Por otro, resulta útil generar gráficas que permitan analizar métricas como confianza media, estabilidad temporal, pérdidas de keypoints o número de anomalías por secuencia.

Para la visualización directa sobre el vídeo, OpenCV permite dibujar puntos, líneas, textos y otros elementos gráficos sobre los frames. Esta funcionalidad será útil para representar los keypoints y las conexiones del esqueleto. Además, permitirá marcar visualmente frames o puntos problemáticos cuando se detecten inconsistencias en la salida del modelo.

Para la representación de métricas y resultados agregados se considera el uso de Matplotlib. Matplotlib es una biblioteca de visualización ampliamente utilizada en Python, especialmente para generar gráficas 2D, diagramas y figuras exportables [?]. En el contexto del proyecto, puede utilizarse para representar la evolución de la confianza, la variación de keypoints a lo largo del tiempo o comparativas entre modelos.

La combinación de visualización sobre vídeo y gráficas de análisis aporta una visión más completa del comportamiento de los modelos. La primera permite comprobar los resultados de forma intuitiva, mientras que la segunda facilita interpretar tendencias y comparar métricas de manera más objetiva.

4.9. Empaquetado y distribución de la aplicación

Aunque el objetivo principal del TFG no es distribuir comercialmente el programa, sí resulta interesante considerar cómo podría ejecutarse la aplicación en un equipo sin necesidad de lanzar manualmente los scripts desde el entorno de desarrollo. Para ello, una herramienta posible es PyInstaller.

PyInstaller permite empaquetar aplicaciones Python junto con sus dependencias para generar ejecutables en sistemas como Windows, macOS o Linux [?]. Esto puede facilitar la entrega de una versión funcional del programa, especialmente si se desea que el usuario pueda abrir la aplicación de forma más directa.

No obstante, el empaquetado de aplicaciones que utilizan bibliotecas pesadas como OpenCV, TensorFlow, PySide6 o modelos de aprendizaje profundo puede presentar dificultades. Algunos módulos pueden requerir configuraciones adicionales, inclusión manual de archivos o ajustes específicos. Por tanto, PyInstaller se considera una herramienta útil, pero su uso deberá valorarse en función del estado final de la aplicación y de las dependencias realmente integradas.

Como alternativa, también puede entregarse el proyecto con instrucciones de instalación y ejecución mediante un entorno virtual de Python. Esta opción puede ser más transparente desde el punto de vista académico, ya que facilita revisar el código, instalar dependencias y reproducir el entorno de desarrollo.

4.10. Comparativa y justificación de herramientas seleccionadas

La elección final de herramientas se basa en un equilibrio entre potencia técnica, facilidad de uso, documentación disponible y adecuación al alcance del proyecto. No se ha buscado únicamente seleccionar las tecnologías más avanzadas, sino aquellas que permiten construir una solución funcional, mantenible y justificable dentro de un Trabajo de Fin de Grado.

Elemento	Alternativas consideradas	Elección principal
Lenguaje	Python, C++, C#	Python, por su ecosistema de IA, visión por computador y análisis de datos.
Entorno	Visual Studio Code, PyCharm, Visual Studio	Visual Studio Code, por su flexibilidad, ligereza y soporte para Python.
Procesamiento de vídeo	OpenCV, herramientas específicas de vídeo	OpenCV, por su integración con Python y uso extendido en visión por computador.
Interfaz gráfica	Tkinter, PySide6, PyQt6, Streamlit, C# WPF	PySide6, por permitir una aplicación de escritorio profesional manteniendo Python.
Modelos de pose	MediaPipe, MoveNet, YOLO-Pose, OpenPose, AlphaPose, HRNet, RTMPose, ViTPose	MediaPipe, MoveNet y YOLO-Pose como candidatos principales por viabilidad de integración.
Análisis de datos	NumPy, Pandas, CSV, JSON	NumPy y Pandas para cálculo y organización; CSV/JSON para exportación.
Visualización	OpenCV, Matplotlib	OpenCV para vídeo y Matplotlib para gráficas de métricas.
Distribución	Ejecución desde entorno Python, PyInstaller	PyInstaller como opción de empaquetado, manteniendo entorno virtual como alternativa.

Tabla 4.1: Resumen de alternativas consideradas y herramientas seleccionadas para el proyecto.

En conclusión, el conjunto de herramientas seleccionado permite abordar todas las partes principales del proyecto: desarrollo de la aplicación, procesamiento de vídeo, ejecución de modelos de detección de pose, análisis técnico de los esqueletos generados, visualización de resultados y exportación de datos. La elección de Python como eje central facilita la integración del sistema y permite mantener una arquitectura coherente y modular.

5. Aspectos relevantes del desarrollo del proyecto

5.1. Introducción

El desarrollo del presente Trabajo Fin de Grado ha supuesto la aplicación práctica de conocimientos adquiridos durante el grado en materias como Ingeniería del Software, Programación, Inteligencia Artificial, Visión por Computador y Desarrollo de Interfaces Gráficas.

A diferencia de otros proyectos centrados únicamente en la implementación de un algoritmo, este trabajo ha requerido el diseño e integración de diferentes componentes software que colaboran entre sí para ofrecer una herramienta funcional destinada al análisis del movimiento humano mediante técnicas de estimación de pose.

Durante el desarrollo no solo se abordó la implementación de la aplicación, sino también la investigación sobre los diferentes modelos existentes, el estudio de sus características, la comparación entre distintas alternativas y el diseño de una arquitectura suficientemente flexible para facilitar la incorporación de nuevos modelos en el futuro.

Este capítulo recoge las principales decisiones adoptadas durante el desarrollo del proyecto, justificando las soluciones implementadas y analizando aquellas dificultades que surgieron durante el proceso de desarrollo.

Como comentario adicional, debo recalcar que este proyecto es fruto de uno anterior que comencé a realizar previamente con los tutores Jose Luis Garrido Labrador y Jose Miguel Ramirez Sanz, profesores que tuvieron una idea como parte de un proyecto conjunto. En esa idea se quería investigar

acercar de las herramientas 3D para el tratamiento clínico del Parkinson, y dado que el tema me resulta de gran interés y, en mi caso, es cercano, no he podido evitar modificar a mi gusto dicho proyecto para poder tener un punto de partida en este tema. La mayoría del proyecto está hecho desde cero, pero un existen librerías y puntos de vista compartidos con el proyecto anterior.

5.2. Metodología de desarrollo

Desde las primeras fases del proyecto se optó por seguir una metodología de desarrollo iterativa e incremental inspirada en los principios de las metodologías ágiles.

En lugar de desarrollar toda la aplicación de forma secuencial, el trabajo se dividió en diferentes etapas o Sprints, cada uno de ellos centrado en alcanzar objetivos concretos. Esta organización permitió validar continuamente el funcionamiento del sistema, detectar errores de forma temprana y modificar determinados aspectos del diseño conforme aumentaba el conocimiento sobre los modelos de estimación de pose.

La planificación inicial estuvo principalmente orientada a la fase de investigación bibliográfica, ya que una parte importante del proyecto consistía en comprender el funcionamiento de las diferentes arquitecturas utilizadas actualmente para la estimación de pose humana.

Posteriormente, una vez identificadas las tecnologías más adecuadas, comenzó el diseño de la arquitectura de la aplicación, priorizando especialmente la modularidad y la independencia entre componentes.

Cada nueva funcionalidad fue implementándose de forma incremental, realizando pruebas funcionales tras cada modificación importante para asegurar la estabilidad del sistema.

Esta forma de trabajo permitió adaptar continuamente la aplicación conforme aparecían nuevas necesidades durante el desarrollo, evitando la aparición de grandes bloques de código difíciles de mantener o modificar.

5.3. Análisis del problema

Uno de los primeros aspectos analizados durante el desarrollo fue determinar qué tipo de aplicación resultaba más adecuada para los objetivos planteados.

Aunque inicialmente se valoró desarrollar una aplicación web, finalmente se optó por una aplicación de escritorio debido a varias razones.

En primer lugar, el procesamiento de vídeo mediante modelos de visión por computador requiere un elevado consumo de recursos computacionales. Ejecutar dichos modelos directamente sobre el equipo del usuario permite aprovechar el hardware disponible sin necesidad de enviar los vídeos a un servidor remoto, reduciendo considerablemente el tiempo de procesamiento y eliminando problemas relacionados con la privacidad de los datos.

En segundo lugar, el proyecto pretende servir como plataforma de experimentación para comparar diferentes modelos de estimación de pose. Una aplicación de escritorio ofrece una mayor flexibilidad para integrar bibliotecas especializadas como OpenCV, PyTorch o TensorFlow, simplificando además el acceso directo al sistema de archivos para cargar y exportar vídeos.

Otro aspecto importante fue definir el tipo de información que debía almacenar la aplicación durante el procesamiento. En lugar de guardar únicamente el resultado visual generado por cada modelo, se decidió almacenar también las coordenadas de todos los keypoints detectados. Esta decisión permite reutilizar posteriormente dichos datos para calcular nuevas métricas biomecánicas sin necesidad de volver a procesar el vídeo completo.

Del mismo modo, se decidió desacoplar completamente la lógica correspondiente a la detección de pose del resto de módulos de la aplicación. Esta decisión facilitará la incorporación de nuevos modelos en futuras versiones, permitiendo realizar comparaciones entre diferentes algoritmos utilizando exactamente el mismo flujo de procesamiento.

5.4. Diseño de la arquitectura software

Uno de los aspectos más relevantes del proyecto fue el diseño de una arquitectura modular que facilitase tanto el mantenimiento del código como la incorporación de nuevas funcionalidades.

Desde el inicio se evitó concentrar toda la lógica de la aplicación en una única clase principal. En su lugar, el proyecto se dividió en diferentes módulos, cada uno responsable de una tarea específica.

La interfaz gráfica quedó completamente separada del procesamiento de vídeo. Mientras que la clase `MainWindow` únicamente gestiona la interacción con el usuario, el resto de funcionalidades fueron encapsuladas en módulos independientes encargados del procesamiento, cálculo de métricas, detección de anomalías y exportación de resultados.

Esta separación reduce considerablemente el acoplamiento entre componentes y mejora la reutilización del código.

Especialmente importante fue el diseño de una clase base común para todos los detectores de pose. Esta decisión permitió definir una interfaz uniforme que cualquier modelo de estimación debe implementar.

Gracias a esta aproximación, el resto de módulos de la aplicación trabajan siempre con la misma estructura de datos, independientemente del modelo utilizado para detectar la pose. Como consecuencia, añadir un nuevo detector únicamente requiere implementar los métodos definidos en la clase base, sin modificar el funcionamiento del resto del sistema.

La arquitectura modular también facilita la realización de pruebas independientes sobre cada componente, simplificando tanto el mantenimiento como la localización de posibles errores durante el desarrollo.

5.5. Diseño de la interfaz gráfica

La interfaz gráfica constituye el principal punto de interacción entre el usuario y la aplicación, por lo que durante su diseño se priorizaron aspectos como la sencillez, la claridad visual y la facilidad de uso. Dado que el proyecto está orientado al análisis de vídeos mediante técnicas de estimación de pose, se consideró fundamental que el usuario pudiera acceder a las funcionalidades principales de forma intuitiva, minimizando el número de acciones necesarias para realizar un análisis completo.

Para el desarrollo de la interfaz se seleccionó la biblioteca *PySide6*, correspondiente a la implementación oficial del framework Qt para Python. Esta elección estuvo motivada por varios factores. En primer lugar, Qt proporciona un conjunto muy amplio de componentes gráficos, permitiendo desarrollar aplicaciones de escritorio modernas y multiplataforma. Además, su integración con Python resulta especialmente sencilla y ofrece una excelente compatibilidad con bibliotecas utilizadas durante el proyecto, como OpenCV o NumPy.

La ventana principal de la aplicación se diseñó siguiendo una distribución basada en paneles, diferenciando claramente la zona destinada a la visualización del vídeo de aquella dedicada a las opciones de configuración y control del procesamiento. Esta organización facilita que el usuario pueda supervisar en todo momento el estado del análisis sin perder de vista la información mostrada por la aplicación.

El área central de la interfaz se reserva para la reproducción del vídeo y la representación del esqueleto humano detectado sobre cada fotograma. Esta decisión permite observar simultáneamente la imagen original y el resultado generado por el modelo de estimación de pose, facilitando la validación visual de las detecciones realizadas.

En uno de los laterales se sitúan los principales controles de la aplicación. Entre ellos se incluyen las opciones para cargar un vídeo desde el sistema de archivos, seleccionar el modelo de estimación de pose, iniciar o detener el procesamiento, reiniciar el análisis y exportar los resultados obtenidos. Agrupar estas funcionalidades en un único panel evita la dispersión de controles por la ventana y mejora la experiencia de usuario.

Otro aspecto considerado durante el diseño fue la presentación de la información obtenida durante el procesamiento. En lugar de mostrar únicamente el vídeo procesado, la interfaz incorpora diferentes elementos destinados a informar del estado del análisis, como indicadores de progreso, estadísticas generales y mensajes relacionados con posibles anomalías detectadas durante la ejecución.

Con el objetivo de mantener una apariencia homogénea en toda la aplicación, se optó por utilizar hojas de estilo (*Qt Style Sheets*), permitiendo personalizar el aspecto visual de los distintos componentes sin modificar la lógica de programación. Gracias a esta separación entre funcionalidad y presentación, resulta sencillo adaptar la apariencia de la aplicación o incorporar nuevos estilos visuales en futuras versiones.

Durante el desarrollo también se tuvo especialmente en cuenta la modularidad de la interfaz. La lógica asociada a la interacción con el usuario quedó encapsulada en la clase `MainWindow`, mientras que las tareas relacionadas con el procesamiento de vídeo, el cálculo de métricas y la detección de pose fueron delegadas en módulos independientes. Esta separación de responsabilidades reduce el acoplamiento entre componentes y facilita considerablemente el mantenimiento del código.

Finalmente, el diseño de la interfaz se realizó pensando en su posible evolución futura. La incorporación de nuevos modelos de estimación de pose, nuevas métricas biomecánicas o nuevos formatos de exportación puede realizarse mediante pequeñas modificaciones sobre los controles existentes, sin necesidad de rediseñar completamente la aplicación. Esta capacidad de ampliación fue uno de los criterios fundamentales durante todo el proceso de desarrollo.

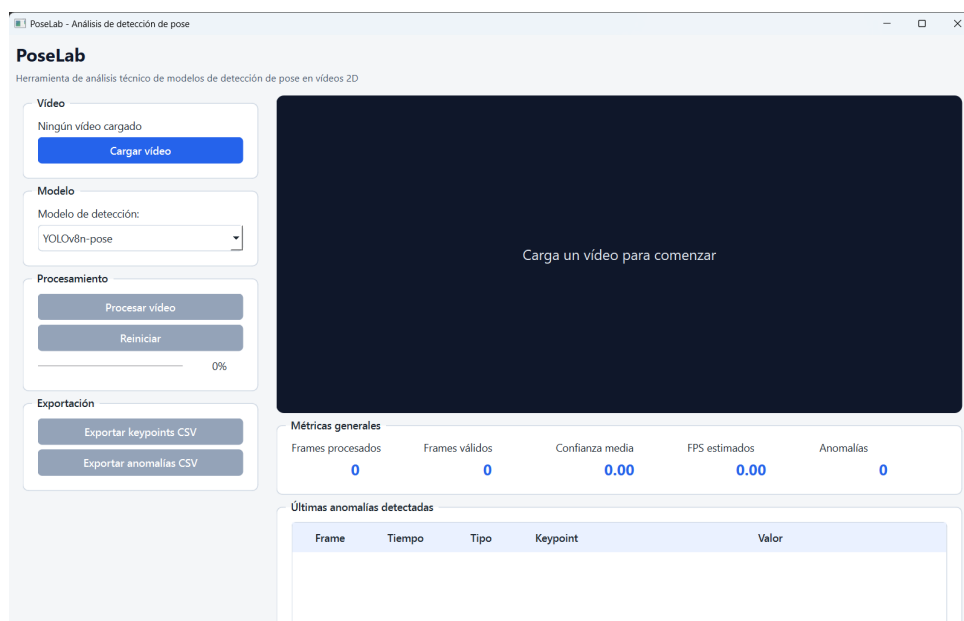


Figura 5.1: Interfaz principal de PoseLab.

5.6. Implementación del procesamiento de vídeo

El procesamiento de vídeo constituye el núcleo funcional de la aplicación, ya que sobre él se apoyan el resto de módulos encargados de la detección de pose, el cálculo de métricas y la generación de resultados. Desde el inicio del desarrollo se consideró prioritario diseñar un sistema que permitiera procesar vídeos de forma eficiente y que, al mismo tiempo, fuese fácilmente adaptable a distintos modelos de estimación de pose.

Para la gestión de los archivos de vídeo se seleccionó la biblioteca OpenCV, ampliamente utilizada en aplicaciones de visión por computador debido a su elevado rendimiento y a la gran cantidad de funcionalidades que proporciona para la manipulación de imágenes y secuencias de vídeo.

Con el objetivo de mantener una correcta separación de responsabilidades, toda la lógica relacionada con la lectura del vídeo se encapsuló dentro de la clase **VideoManager**. Esta clase actúa como intermediario entre la interfaz gráfica y OpenCV, ocultando al resto de módulos los detalles relacionados con la apertura de archivos, la obtención de fotogramas y el control de la reproducción.

5.7. INTEGRACIÓN DE LOS MODELOS DE ESTIMACIÓN DE POSE

Uno de los aspectos más importantes durante el diseño fue evitar cargar el vídeo completo en memoria. En su lugar, la aplicación procesa cada fotograma de forma secuencial conforme avanza la reproducción. Esta estrategia reduce considerablemente el consumo de memoria y permite trabajar con vídeos de larga duración sin comprometer el rendimiento del sistema.

Cada vez que el usuario inicia el procesamiento, el gestor de vídeo obtiene el siguiente fotograma y lo envía al modelo de estimación de pose seleccionado. El resultado generado por dicho modelo contiene tanto la información necesaria para representar el esqueleto humano como las coordenadas de los distintos puntos articulares detectados.

Una vez procesado el fotograma, éste se representa inmediatamente sobre la interfaz gráfica junto con el esqueleto superpuesto. Paralelamente, la información correspondiente a los keypoints se almacena temporalmente para ser utilizada posteriormente durante el cálculo de métricas biomecánicas y la exportación de resultados.

Para controlar el ritmo de procesamiento se utilizó un temporizador proporcionado por Qt (QTimer), encargado de ejecutar periódicamente el procesamiento del siguiente fotograma. Gracias a este mecanismo, la interfaz permanece completamente operativa durante toda la ejecución, evitando bloqueos o pérdidas de respuesta que podrían producirse si el procesamiento se realizara mediante un único bucle secuencial.

Otro aspecto especialmente relevante fue la sincronización entre el índice del fotograma y el instante temporal asociado. Cada fotograma dispone de un identificador único y de una marca temporal calculada a partir de la velocidad de reproducción del vídeo (FPS). Esta información resulta imprescindible para calcular posteriormente determinadas métricas relacionadas con la evolución temporal del movimiento.

Finalmente, una vez alcanzado el último fotograma del vídeo, el sistema libera automáticamente todos los recursos asociados al archivo abierto, garantizando un uso eficiente de la memoria y evitando posibles fugas de recursos durante la ejecución continuada de la aplicación.

5.7. Integración de los modelos de estimación de pose

Uno de los principales objetivos del proyecto consistía en desarrollar una aplicación capaz de comparar diferentes modelos de estimación de

pose utilizando una misma arquitectura de procesamiento. Para conseguirlo fue necesario diseñar un mecanismo que desacoplase completamente el funcionamiento interno de cada modelo del resto de componentes de la aplicación.

En lugar de implementar cada modelo de forma independiente dentro de la interfaz gráfica, se optó por definir una clase base común denominada **BasePoseDetector**. Esta clase establece una interfaz uniforme que todos los detectores deben implementar, independientemente de la tecnología utilizada para realizar la detección.

Cada modelo únicamente debe proporcionar los métodos necesarios para inicializar el detector, procesar un fotograma, representar gráficamente los keypoints obtenidos y liberar correctamente los recursos utilizados durante la ejecución.

Gracias a esta aproximación, el resto de módulos trabajan siempre con la misma estructura de datos, sin necesidad de conocer qué modelo concreto está realizando la inferencia. Esta independencia facilita enormemente el mantenimiento del sistema y permite incorporar nuevos detectores con modificaciones mínimas sobre el código existente.

La información obtenida durante la inferencia se transforma inmediatamente a una estructura de datos común basada en objetos **Keypoint** y **PoseFrameResult**. Esta decisión evita que las diferencias existentes entre las distintas bibliotecas afecten al resto del sistema, permitiendo reutilizar exactamente los mismos algoritmos de cálculo de métricas, detección de anomalías y exportación de resultados.

Otro aspecto importante durante esta fase fue la normalización de las coordenadas articulares. Aunque algunos modelos devuelven coordenadas en píxeles y otros emplean coordenadas normalizadas respecto al tamaño de la imagen, todos los resultados fueron convertidos a un formato homogéneo antes de ser procesados por el resto de módulos de la aplicación.

Esta estrategia proporciona una arquitectura altamente escalable y preparada para futuras ampliaciones, permitiendo incorporar nuevos modelos de estimación de pose sin modificar la lógica general del sistema.

5.8. Procesamiento y análisis de los keypoints

Los keypoints obtenidos durante la inferencia constituyen el elemento principal sobre el que se desarrolla el análisis biomecánico implementado por la aplicación.

Cada keypoint representa la posición estimada de una articulación concreta del cuerpo humano y se almacena junto con información adicional como el fotograma de origen, el instante temporal, el modelo utilizado y el nivel de confianza asociado a la detección.

En lugar de utilizar directamente las estructuras proporcionadas por cada biblioteca de visión por computador, se decidió definir un modelo de datos propio que unificase la representación de toda la información generada durante el procesamiento. Esta decisión simplifica considerablemente el resto del desarrollo y facilita la reutilización del código entre diferentes detectores.

A partir de estas coordenadas se calculan diferentes métricas relacionadas con el movimiento observado. Entre ellas destacan el tiempo medio de procesamiento por fotograma, el porcentaje de fotogramas válidos, la confianza media de las detecciones y otros indicadores que permiten comparar objetivamente el comportamiento de los distintos modelos evaluados.

Paralelamente, la aplicación incorpora un módulo específico encargado de detectar posibles anomalías durante el procesamiento. Estas anomalías incluyen situaciones como keypoints con niveles de confianza excesivamente bajos, desaparición temporal de determinadas articulaciones o inconsistencias espaciales entre fotogramas consecutivos.

La separación entre el módulo de detección y el módulo de análisis permite ampliar fácilmente el conjunto de métricas calculadas sin modificar el funcionamiento de los detectores de pose, favoreciendo así la escalabilidad del sistema.

5.9. Exportación de resultados

Una vez finalizado el procesamiento del vídeo, la aplicación ofrece la posibilidad de exportar la información obtenida para facilitar su posterior análisis mediante otras herramientas estadísticas o de investigación.

Desde las primeras fases del desarrollo se consideró que el resultado del procesamiento no debía limitarse únicamente a la representación gráfica del

esqueleto sobre el vídeo. El verdadero valor de la aplicación reside en la información generada durante el análisis, especialmente en las coordenadas articulares, las métricas calculadas y las posibles anomalías detectadas.

Por este motivo se implementó un módulo independiente encargado exclusivamente de la exportación de resultados. Esta separación evita mezclar la lógica de almacenamiento con el procesamiento principal y facilita la incorporación de nuevos formatos de salida en futuras versiones.

Inicialmente se decidió utilizar el formato CSV para almacenar los keypoints detectados, debido a su amplia compatibilidad con hojas de cálculo, software estadístico y lenguajes de programación como Python, R o MATLAB. Cada fila representa la información correspondiente a una articulación detectada en un fotograma concreto, incluyendo datos como el identificador del fotograma, el instante temporal, el nombre del keypoint, sus coordenadas y el nivel de confianza asociado.

Además del formato CSV, la aplicación incorpora la posibilidad de exportar determinados resultados en formato JSON. Este formato resulta especialmente útil cuando se desea intercambiar información entre diferentes aplicaciones o almacenar estructuras de datos más complejas conservando su organización jerárquica.

La existencia de un módulo de exportación independiente permite ampliar fácilmente la aplicación para incorporar nuevos formatos de salida, como bases de datos, hojas de cálculo Excel o informes PDF, sin necesidad de modificar el resto de componentes del sistema.

5.10. Dificultades encontradas durante el desarrollo

El desarrollo del proyecto estuvo acompañado de diversas dificultades técnicas que obligaron a replantear determinadas decisiones de diseño e implementación.

Uno de los principales retos fue la integración de diferentes modelos de estimación de pose dentro de una misma aplicación. Cada biblioteca utilizada presenta una interfaz propia, genera estructuras de datos diferentes y utiliza distintos formatos para representar los keypoints detectados. Esta situación hizo necesario diseñar una capa de abstracción que unificara toda la información generada durante la inferencia.

Otra dificultad importante estuvo relacionada con la gestión de las dependencias externas. Bibliotecas como OpenCV, PyTorch o determinados modelos de estimación de pose requieren configuraciones específicas y presentan incompatibilidades entre algunas versiones, especialmente en sistemas Windows. Durante el desarrollo fue necesario realizar diferentes pruebas para identificar aquellas combinaciones de dependencias que ofrecían un funcionamiento estable.

Problemas de compatibilidad con MediaPipe

Una de las primeras alternativas planteadas para la estimación de pose fue MediaPipe, ya que se trata de una biblioteca ampliamente utilizada para detectar puntos corporales en imágenes y vídeos. Su principal ventaja era que permitía obtener directamente un conjunto de keypoints del cuerpo humano y dibujar el esqueleto sobre cada fotograma, lo que encajaba muy bien con los objetivos iniciales del proyecto.

Sin embargo, durante la integración aparecieron diferentes problemas de compatibilidad. En primer lugar, la aplicación se quedaba bloqueada al importar el detector, concretamente al ejecutar la línea encargada de importar el módulo desarrollado para MediaPipe. Para localizar el origen del problema se añadieron mensajes de depuración en el archivo principal y en la ventana principal de la aplicación. De esta forma se pudo comprobar que el programa se detenía durante la importación de MediaPipe y no durante la carga del vídeo o la creación de la interfaz.

Se realizaron diferentes pruebas desde la terminal para comprobar si la biblioteca estaba instalada correctamente:

```
.venv\Scripts\python.exe -m pip show mediapipe
.venv\Scripts\python.exe -c "import mediapipe as mp; print(mp.__version__)"
```

Aunque MediaPipe aparecía instalado, algunas versiones recientes no mantenían el mismo acceso a la API clásica utilizada en muchos ejemplos, basada en:

```
mp.solutions.pose
```

En una de las pruebas se obtuvo un error indicando que el módulo `mediapipe` no tenía el atributo `solutions`, lo que impedía utilizar el código previsto originalmente. También se intentó forzar una versión anterior de MediaPipe junto con versiones concretas de OpenCV y Protobuf:

```
.venv\Scripts\python.exe -m pip uninstall mediapipe opencv-python opencv-contrib-python
.venv\Scripts\python.exe -m pip install mediapipe==0.10.14 opencv-python==4.10.0.84
```

A pesar de estas pruebas, el detector seguía presentando bloqueos durante su inicialización. El problema era especialmente difícil de diagnosticar porque en ocasiones no se producía una excepción controlada de Python, sino que la aplicación simplemente dejaba de responder o se cerraba durante la creación del detector.

Por este motivo, se concluyó que MediaPipe introducía una dependencia demasiado inestable para el entorno de desarrollo utilizado, especialmente teniendo en cuenta que el objetivo principal era disponer de una aplicación funcional y defendible dentro del alcance temporal del Trabajo Fin de Grado.

Problemas de compatibilidad con OpenCV

Durante el proceso de instalación y prueba de MediaPipe también se detectaron conflictos relacionados con OpenCV. Inicialmente se esperaba utilizar la versión:

```
opencv-python==4.10.0.84
```

Sin embargo, al comprobar la versión realmente cargada por Python se obtuvo un resultado diferente:

```
.venv\Scripts\python.exe -c "import cv2; print(cv2.__version__)"
```

El resultado mostraba una versión 5.0.0, lo que no coincidía con la versión declarada en el archivo `requirements.txt`. Para analizar el problema se ejecutó:

```
.venv\Scripts\python.exe -m pip show opencv-python opencv-contrib-python opencv-python-headless
```

Con este comando se comprobó que coexistían dos paquetes distintos de OpenCV dentro del mismo entorno virtual: `opencv-python` y `opencv-contrib-python`. El segundo había sido instalado como dependencia de MediaPipe y estaba provocando que Python importase una versión distinta de `cv2`. Este tipo de conflicto es problemático porque dos paquetes diferentes proporcionan el mismo módulo de importación, generando comportamientos poco predecibles.

Para intentar corregirlo se eliminó OpenCV por completo y se reinstaló únicamente la versión necesaria:

```
.venv\Scripts\python.exe -m pip uninstall opencv-python opencv-contrib-python opencv-python-headless  
.venv\Scripts\python.exe -m pip install opencv-python==4.10.0.84
```

Tras esta limpieza se comprobó de nuevo la versión instalada. Aunque el conflicto con OpenCV pudo solucionarse, quedó claro que la combinación de MediaPipe con otras dependencias añadía complejidad al entorno y aumentaba el riesgo de incompatibilidades.

Problemas de inicialización con PyTorch y YOLO

Como alternativa a MediaPipe se planteó la integración de modelos YOLO de estimación de pose mediante la biblioteca Ultralytics. Esta opción resultaba especialmente interesante porque permitía utilizar modelos como `yolov8n-pose.pt` o `yolo11n-pose.pt`, capaces de detectar keypoints corporales y generar una representación visual del esqueleto.

La instalación se realizó mediante:

```
.venv\Scripts\python.exe -m pip install ultralytics
```

Después se verificó que Ultralytics podía importarse correctamente:

```
.venv\Scripts\python.exe -c "from ultralytics import YOLO; print('OK!')"
```

Esta prueba funcionaba, por lo que inicialmente parecía que la biblioteca estaba correctamente instalada. También se comprobó la instalación de PyTorch:

```
.venv\Scripts\python.exe -c "import torch; print(torch.__version__)"
```

El problema apareció al intentar crear el modelo desde la aplicación gráfica. Al pulsar el botón de procesamiento, el sistema mostraba el mensaje de depuración:

Creando detector...

pero la aplicación se cerraba o quedaba bloqueada inmediatamente después. Posteriormente se obtuvo un error más concreto relacionado con una biblioteca dinámica de Windows:

```
[WinError 1114] Error en una rutina de inicialización de biblioteca de
vínculos dinámicos (DLL). Error loading:
torch\lib\c10.dll
```

Este error indicaba que PyTorch no podía inicializar correctamente una de sus bibliotecas internas. En Windows, este tipo de errores suele estar relacionado con incompatibilidades entre versiones, dependencias nativas, redistribuibles de Microsoft Visual C++ o problemas con la instalación de paquetes compilados.

Para intentar resolverlo se realizó una reinstalación limpia de PyTorch en versión CPU, evitando dependencias CUDA innecesarias:

```
.venv\Scripts\python.exe -m pip uninstall torch torchvision torchaudio ultra
.venv\Scripts\python.exe -m pip cache purge
.venv\Scripts\python.exe -m pip install torch torchvision torchaudio --index
.venv\Scripts\python.exe -m pip install ultralytics
```

También se probaron versiones concretas de PyTorch para reducir la posibilidad de incompatibilidades:

```
.venv\Scripts\python.exe -m pip install torch==2.5.1 torchvision==0.20.1 tor
```

Aunque las pruebas aisladas de importación funcionaban, la creación del detector dentro de la aplicación seguía dando problemas en el entorno de desarrollo. Esto confirmó que el fallo no estaba en la lógica principal de la aplicación, sino en la inicialización interna de PyTorch y sus bibliotecas nativas.

Depuración realizada durante la integración de modelos

Para localizar el punto exacto de fallo se introdujeron mensajes de depuración dentro del flujo de creación del detector. En concreto, se añadieron trazas en el método encargado de iniciar el procesamiento:

5.10. DIFICULTADES ENCONTRADAS DURANTE EL DESARROLLO

```
print("Creando detector...")
self.detector = self.create_detector()
print("Detector creado correctamente")
```

Esta depuración permitió comprobar que el programa llegaba hasta la creación del detector, pero nunca alcanzaba el mensaje posterior. Por tanto, el error se encontraba en la inicialización de la biblioteca externa utilizada por el modelo, y no en la carga del vídeo, la interfaz gráfica o el procesamiento de fotogramas.

También se añadieron bloques `try/except` para capturar errores de Python y mostrarlos mediante cuadros de diálogo de la interfaz:

```
try:
    if self.detector is None:
        print("Creando detector...")
        self.detector = self.create_detector()
        print("Detector creado correctamente")
except Exception as error:
    import traceback
    traceback.print_exc()
    QMessageBox.critical(self, "Error creando detector", str(error))
```

Esta estrategia permitió obtener información más clara sobre algunos errores, como la ausencia de módulos o conflictos de versiones. No obstante, en determinados casos el fallo se producía a nivel de biblioteca nativa, por lo que no siempre podía ser capturado correctamente mediante excepciones de Python.

Decisión de implementar una solución básica basada en OpenCV

Debido a los problemas encontrados con MediaPipe, PyTorch y Ultralytics, se tomó la decisión de implementar una solución alternativa más sencilla y estable basada únicamente en OpenCV. Esta decisión se adoptó para garantizar que la aplicación pudiera ejecutarse correctamente y cumplir al menos con el flujo principal definido en los objetivos: cargar un vídeo, procesarlo fotograma a fotograma, visualizar información sobre el movimiento y exportar resultados.

La solución final se basó en un detector simple de movimiento. En lugar de utilizar un modelo profundo de estimación de pose, se comparan fotogramas consecutivos para identificar las regiones de la imagen en las que existe movimiento. A partir de la región detectada se generan puntos aproximados que permiten mantener la estructura interna de la aplicación basada en keypoints.

Esta alternativa no pretende alcanzar la precisión de un modelo real de estimación de pose, pero permitió conservar la arquitectura modular diseñada originalmente. El sistema sigue utilizando clases como `BasePoseDetector`, `Keypoint` y `PoseFrameResult`, por lo que en el futuro sería posible sustituir el detector básico por un modelo avanzado cuando el entorno de dependencias sea estable.

La principal ventaja de esta solución es que reduce las dependencias del proyecto a un conjunto mucho más controlado:

```
opencv-python==4.10.0.84
numpy==1.26.4
pandas==2.2.2
PySide6==6.7.2
matplotlib==3.9.1
```

Además, permite eliminar dependencias problemáticas como:

```
mediapipe
torch
torchvision
torchaudio
ultralytics
opencv-contrib-python
```

La limpieza del entorno se realizó mediante:

```
.venv\Scripts\python.exe -m pip uninstall torch torchvision torchaudio ultra
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Con esta modificación, la aplicación mantiene su valor como prototipo funcional de análisis de movimiento. Aunque la detección implementada es más básica, el sistema conserva las partes esenciales del proyecto: gestión

de vídeo, procesamiento frame a frame, visualización de resultados, cálculo de métricas, detección de anomalías, exportación y arquitectura preparada para incorporar modelos más avanzados en líneas futuras.

Valoración de la solución adoptada

La decisión de sustituir temporalmente los modelos avanzados por un detector básico fue una medida pragmática motivada por las limitaciones del entorno y el tiempo disponible. En un proyecto académico es importante no solo intentar integrar las tecnologías más avanzadas, sino también ser capaz de tomar decisiones razonadas cuando una dependencia externa compromete la estabilidad del sistema.

Desde el punto de vista de ingeniería del software, la solución adoptada permitió preservar el diseño modular del proyecto y asegurar que la aplicación pudiera ejecutarse de principio a fin. Además, deja claramente identificada una línea futura de mejora: sustituir el detector básico por modelos de estimación de pose como YOLO-Pose, MoveNet o MediaPipe cuando se disponga de un entorno de ejecución compatible y estable.

Por tanto, aunque la solución final es más sencilla que la planteada inicialmente, se considera adecuada dentro del alcance del Trabajo Fin de Grado, ya que permite demostrar el funcionamiento completo de la arquitectura y documentar de forma realista las dificultades encontradas durante la integración de bibliotecas de inteligencia artificial en sistemas Windows.

Finalmente, la validación del sistema requirió realizar numerosas pruebas utilizando diferentes vídeos y condiciones de grabación. Estas pruebas permitieron detectar pequeños errores relacionados con la carga de archivos, el tratamiento de determinados fotogramas y la gestión de excepciones, contribuyendo a mejorar la estabilidad general de la aplicación.

5.11. Valoración del desarrollo realizado

El desarrollo de este Trabajo Fin de Grado ha permitido aplicar de forma integrada numerosos conocimientos adquiridos a lo largo del Grado en Ingeniería Informática, combinando aspectos relacionados con la ingeniería del software, la visión por computador, el procesamiento digital de imágenes y el desarrollo de interfaces gráficas.

Más allá de la implementación técnica, uno de los aprendizajes más importantes ha sido comprender la importancia del diseño de una arquitect-

tura software adecuada. La modularidad conseguida durante el desarrollo facilita el mantenimiento del sistema y permite incorporar nuevos modelos de estimación de pose con un impacto mínimo sobre el resto de componentes de la aplicación.

Igualmente, el proyecto ha puesto de manifiesto la complejidad asociada al desarrollo de aplicaciones que integran bibliotecas externas de inteligencia artificial. La correcta gestión de dependencias, la compatibilidad entre versiones y la adaptación a diferentes plataformas constituyen aspectos tan relevantes como la propia implementación de los algoritmos.

Desde un punto de vista funcional, la aplicación desarrollada cumple los objetivos inicialmente planteados, proporcionando una plataforma capaz de procesar vídeos, representar la pose humana detectada, calcular diferentes métricas relacionadas con el movimiento y exportar los resultados obtenidos para su posterior análisis.

No obstante, el proyecto presenta un importante potencial de crecimiento. La arquitectura diseñada facilita la incorporación de nuevos modelos de estimación de pose, la implementación de métricas biomecánicas más avanzadas y la integración con otras herramientas orientadas al análisis clínico o deportivo. Si se dispusiera de una CPU totalmente preparada para ello, el código está diseñado para poder interpretar todas aquellas librerías que se deseen modificando únicamente la clase `main-window.py` que es la encargada, entre otras muchas cosas de habilitar el scroll que permitiría utilizar diferentes librerías de modelos de análisis de pose.

En conjunto, el desarrollo del proyecto ha supuesto una experiencia especialmente enriquecedora tanto desde el punto de vista técnico como metodológico, permitiendo afrontar problemas reales relacionados con el diseño, implementación y validación de aplicaciones basadas en inteligencia artificial. Asimismo, la experiencia adquirida durante este proceso constituye una base sólida para futuros desarrollos relacionados con el análisis automático del movimiento humano y la aplicación de técnicas de visión por computador en el ámbito sanitario.

6. Trabajos relacionados

En este capítulo se presenta un breve repaso de trabajos, modelos y líneas de investigación relacionados con el presente proyecto. El objetivo no es realizar un estado del arte exhaustivo, sino contextualizar el trabajo dentro del campo de la detección de pose humana, el análisis automático del movimiento y la aplicación de técnicas de visión por computador en entornos terapéuticos o de rehabilitación.

El proyecto se apoya en avances previos dentro de la estimación de pose humana mediante aprendizaje profundo. Esta área ha evolucionado de forma significativa en los últimos años, pasando de modelos centrados en imágenes individuales y escenarios controlados a sistemas capaces de trabajar en tiempo real, detectar varias personas y generar representaciones estructuradas del cuerpo humano a partir de vídeos 2D.

6.1. Estimación de pose humana mediante aprendizaje profundo

Uno de los primeros trabajos relevantes en la aplicación de redes profundas a la estimación de pose humana fue DeepPose, propuesto por Toshev y Szegedy. Este enfoque planteaba la detección de pose como un problema de regresión directa de coordenadas corporales a partir de una imagen [18]. Aunque posteriormente surgieron métodos más precisos y robustos, DeepPose supuso un paso importante al demostrar que las redes neuronales profundas podían aplicarse de forma efectiva a la localización de articulaciones humanas.

Más adelante, OpenPose se convirtió en una de las referencias más conocidas dentro de la detección de pose multipersona. Su principal aportación fue el uso de campos de afinidad de partes, o *Part Affinity Fields*, para asociar puntos corporales pertenecientes a una misma persona [2]. Este enfoque permitió trabajar con varias personas en una misma imagen y generó una base muy utilizada en proyectos de investigación, análisis de movimiento e interacción persona-ordenador.

Otro trabajo relevante es AlphaPose, orientado también a la estimación de pose multipersona. A diferencia de los enfoques *bottom-up*, AlphaPose sigue una estrategia *top-down*, detectando primero a las personas y estimando después la pose de cada una de ellas [3]. Este tipo de aproximación ha mostrado buenos resultados en escenarios complejos, aunque suele depender de la calidad del detector de personas utilizado en la primera fase.

También destaca HRNet, una arquitectura diseñada para mantener representaciones de alta resolución durante todo el proceso de estimación [15]. Esta característica resulta especialmente útil en tareas donde la localización precisa de los puntos corporales es importante, ya que evita perder información espacial en las capas intermedias de la red. Aunque su integración práctica puede ser más compleja que la de otros modelos más ligeros, HRNet representa una línea de trabajo relevante en la mejora de precisión de la detección de pose.

6.2. Modelos modernos orientados a eficiencia e integración

Además de los modelos clásicos de investigación, en los últimos años han aparecido soluciones orientadas a facilitar la integración en aplicaciones reales. MediaPipe Pose, desarrollado dentro del ecosistema MediaPipe de Google, permite estimar landmarks corporales de forma eficiente y con una API accesible para aplicaciones prácticas [5]. Su capacidad para trabajar en tiempo real y su conjunto ampliado de puntos corporales lo convierten en una alternativa especialmente interesante para proyectos de análisis de vídeo.

MoveNet, disponible a través de TensorFlow Hub, es otro modelo orientado a la detección rápida de pose humana. Trabaja con 17 puntos corporales y ofrece variantes que permiten priorizar velocidad o precisión [17]. Esta característica lo hace adecuado para comparativas donde se desee estu-

diar el compromiso entre rendimiento computacional y estabilidad de las detecciones.

Por otro lado, los modelos YOLO aplicados a estimación de pose han ganado relevancia por su velocidad y facilidad de uso. Herramientas como Ultralytics permiten utilizar modelos de pose basados en YOLO de forma relativamente directa, combinando detección de personas y estimación de keypoints [19]. Esta aproximación resulta especialmente útil cuando se busca procesar vídeo de manera eficiente dentro de una aplicación.

También se han desarrollado modelos más recientes como RTMPose, orientado a la estimación multipersona en tiempo real dentro del ecosistema MMPose [8], o ViTPose, que incorpora transformadores visuales para la estimación de pose humana [20]. Estos trabajos muestran la evolución del campo hacia arquitecturas más flexibles, precisas y adaptadas a distintos contextos de uso.

6.3. Análisis de movimiento y aplicaciones terapéuticas

La detección de pose humana no solo se ha utilizado en visión por computador general, sino también en aplicaciones relacionadas con el análisis del movimiento humano. En contextos deportivos, terapéuticos o de rehabilitación, la estimación de keypoints permite obtener una representación simplificada del cuerpo y estudiar la evolución de determinadas articulaciones a lo largo del tiempo.

En el ámbito de la rehabilitación, algunos trabajos han explorado el uso de modelos de pose para analizar ejercicios físicos, estimar ángulos articulares o comprobar la ejecución de movimientos. Estas aproximaciones resultan interesantes porque permiten trabajar con vídeos convencionales, reduciendo la necesidad de utilizar sensores físicos, marcadores ópticos o sistemas de captura de movimiento más costosos.

No obstante, es importante distinguir entre el análisis clínico del movimiento y la evaluación técnica de los modelos. El presente proyecto no pretende determinar si un ejercicio terapéutico está bien o mal ejecutado desde el punto de vista médico, sino estudiar la fiabilidad de los modelos de pose cuando se aplican a vídeos de este tipo. Por ello, el interés principal se centra en la estabilidad de los keypoints, la coherencia de los esqueletos generados y la detección de posibles errores técnicos en las salidas del modelo.

Este enfoque conecta con trabajos recientes centrados en la detección de anomalías sobre datos de pose humana. Algunos métodos analizan secuencias de keypoints para identificar patrones atípicos, inconsistencias o desviaciones respecto a una distribución esperada [7]. Aunque este tipo de técnicas puede aplicarse a distintos escenarios, en este proyecto se adopta una interpretación más concreta: detectar anomalías como errores de estimación del modelo, tales como saltos bruscos de puntos, pérdidas temporales o deformaciones artificiales del esqueleto.

6.4. Relación con el proyecto desarrollado

El presente Trabajo de Fin de Grado se sitúa entre estas líneas de trabajo. Por un lado, toma como base los avances en detección de pose humana mediante aprendizaje profundo. Por otro, aplica estos modelos al análisis de vídeos 2D relacionados con contextos médicos, terapéuticos o de rehabilitación. Finalmente, introduce una capa de evaluación técnica orientada a estudiar la calidad de los esqueletos generados.

A diferencia de trabajos centrados únicamente en proponer una nueva arquitectura de red neuronal, este proyecto no busca diseñar un modelo desde cero, sino comparar modelos existentes y valorar su comportamiento dentro de una aplicación práctica. Esta decisión resulta adecuada para el alcance de un TFG, ya que permite centrarse en la integración, evaluación y análisis de herramientas reales.

La principal aportación del proyecto consiste en combinar tres elementos: selección y comparación de modelos de pose, procesamiento de vídeos terapéuticos o neuromotores, y detección de inconsistencias técnicas en los esqueletos generados. De esta forma, el trabajo no se limita a mostrar visualmente los keypoints, sino que busca analizar hasta qué punto las salidas producidas por los modelos son estables, coherentes y útiles para una aplicación software.

En conclusión, los trabajos relacionados muestran que la detección de pose humana es un campo consolidado y en evolución, con aplicaciones cada vez más amplias. El proyecto se apoya en estos avances para desarrollar una herramienta orientada al análisis técnico de modelos de pose en vídeos 2D, manteniendo una perspectiva informática y evitando atribuir al sistema funciones clínicas que no forman parte de su alcance.

7. Conclusiones y Líneas de trabajo futuras

En este capítulo se recogen las principales conclusiones derivadas del planteamiento y desarrollo del proyecto, así como una serie de posibles líneas de trabajo futuras que permitirían ampliar o mejorar el sistema propuesto. El objetivo de este apartado es valorar el grado de cumplimiento de los objetivos iniciales, reflexionar sobre las decisiones técnicas adoptadas y señalar aquellos aspectos que podrían evolucionar en fases posteriores.

7.1. Conclusiones

El presente Trabajo de Fin de Grado se ha planteado como un proyecto de carácter principalmente informático, centrado en el estudio y comparación de modelos de detección de pose humana aplicados a vídeos 2D de carácter médico, terapéutico o de rehabilitación. Aunque el contexto de aplicación se ha relacionado con enfermedades neuromotoras como la esclerosis lateral amiotrófica, el proyecto no ha buscado realizar diagnóstico médico ni valoración clínica, sino analizar la calidad técnica de los esqueletos generados por distintos modelos de visión por computador.

Una de las principales conclusiones del trabajo es que la detección de pose humana constituye una herramienta muy interesante para el análisis automático de movimiento a partir de vídeo. Mediante la estimación de keypoints corporales es posible transformar una secuencia visual compleja en una representación estructurada, más sencilla de procesar y comparar. Esta representación permite estudiar aspectos como la estabilidad temporal

de los puntos, la confianza de las detecciones, la pérdida de articulaciones o la coherencia espacial del esqueleto generado.

También se ha comprobado que no todos los modelos de detección de pose están orientados al mismo tipo de uso. Algunos modelos priorizan la precisión, otros la velocidad de inferencia, otros la facilidad de integración y otros la detección multipersona. Por ello, una parte importante del proyecto ha consistido en analizar diferentes alternativas y justificar cuáles resultan más adecuadas para una aplicación práctica. En este sentido, modelos como MediaPipe Pose, MoveNet y YOLO-Pose se presentan como opciones especialmente interesantes por su equilibrio entre rendimiento, documentación disponible y viabilidad de integración.

Otra conclusión relevante es la importancia de normalizar las salidas de los modelos. Cada herramienta puede devolver un número diferente de puntos corporales, estructuras de datos distintas y valores de confianza con interpretaciones no siempre equivalentes. Por este motivo, resulta necesario definir una representación común de keypoints que permita almacenar, comparar y analizar los resultados de forma homogénea. Esta decisión es fundamental para que el sistema sea modular y pueda ampliarse en el futuro con nuevos modelos.

El proyecto también ha puesto de manifiesto la utilidad de analizar anomalías técnicas en los esqueletos generados. En este trabajo, dichas anomalías no se entienden como alteraciones médicas del sujeto, sino como errores o inconsistencias producidas por el modelo de detección de pose. Saltos bruscos entre frames, pérdidas temporales de puntos, bajas confianzas o deformaciones artificiales del esqueleto son ejemplos de problemas que pueden afectar a la fiabilidad del análisis. Incorporar mecanismos para detectar este tipo de situaciones aporta valor al sistema, ya que permite evaluar no solo si un modelo detecta una pose, sino también si la mantiene de forma estable y coherente a lo largo del vídeo.

Desde el punto de vista técnico, la elección de Python como lenguaje principal resulta adecuada por la disponibilidad de bibliotecas especializadas en visión por computador, aprendizaje profundo, análisis de datos e interfaces gráficas. Herramientas como OpenCV, PySide6, NumPy, Pandas y Matplotlib permiten cubrir las necesidades principales del proyecto, desde la lectura de vídeo hasta la visualización y exportación de resultados. Además, el uso de una arquitectura modular facilita la separación entre procesamiento de vídeo, detección de pose, análisis de keypoints, detección de inconsistencias e interfaz de usuario.

En conjunto, el proyecto permite concluir que es viable diseñar una aplicación de escritorio orientada a comparar modelos de detección de pose en vídeos 2D, mostrando sus resultados de forma visual y calculando métricas técnicas sobre la calidad de las detecciones. Aunque el sistema no sustituye herramientas clínicas ni modelos validados médicamente, sí puede servir como base para estudiar el comportamiento de modelos de visión artificial en contextos terapéuticos o neuromotores.

7.2. Líneas de trabajo futuras

A partir del trabajo realizado, se pueden plantear varias líneas de mejora y ampliación. La primera de ellas sería integrar un mayor número de modelos de detección de pose. Aunque MediaPipe Pose, MoveNet y YOLO-Pose son candidatos adecuados para una primera versión, modelos como OpenPose, AlphaPose, HRNet, RTMPose o ViTPose podrían incorporarse en fases posteriores para realizar una comparación más amplia y completa.

Otra línea de trabajo importante sería mejorar la detección de anomalías técnicas en los esqueletos generados. En una primera aproximación, estas anomalías pueden detectarse mediante reglas basadas en umbrales de desplazamiento, confianza o variación de distancias entre articulaciones. Sin embargo, en versiones futuras podrían incorporarse métodos estadísticos o modelos de aprendizaje automático capaces de identificar patrones de error más complejos en las secuencias de keypoints.

También sería interesante ampliar el conjunto de vídeos utilizados para la evaluación. Cuanto mayor sea la variedad de vídeos, más fiable será la comparación entre modelos. En este sentido, podrían buscarse nuevas bases de datos de rehabilitación, marcha humana, ejercicios terapéuticos o movimientos asociados a enfermedades neuromotoras. La inclusión de vídeos con diferentes condiciones de iluminación, cámara, resolución, postura y movimiento permitiría estudiar mejor la robustez de cada modelo.

Otra posible mejora consistiría en incorporar anotaciones manuales o semiautomáticas en algunos vídeos. Esto permitiría comparar las predicciones de los modelos con una referencia más objetiva y utilizar métricas clásicas de estimación de pose, como PCK, AP u OKS. Aunque esta tarea requiere tiempo, aportaría una evaluación más rigurosa de la precisión de cada modelo.

Desde el punto de vista de la aplicación, una línea futura sería mejorar la interfaz gráfica para hacerla más completa y cómoda de utilizar. Se podrían

añadir paneles de comparación entre modelos, gráficas interactivas, filtros por keypoint, reproducción sincronizada del vídeo original y procesado, y generación automática de informes. Estas mejoras harían que el programa fuese más útil como herramienta de análisis.

También se podría trabajar en la exportación de resultados. Además de CSV y JSON, el sistema podría generar informes en PDF con capturas, métricas principales, gráficas y resumen de anomalías detectadas. Esto facilitaría la documentación de las pruebas realizadas y permitiría incluir resultados de forma más directa en anexos o informes técnicos.

Finalmente, una línea de trabajo más avanzada sería estudiar el uso de información temporal de forma más profunda. En lugar de analizar únicamente frames individuales o variaciones simples entre frames consecutivos, podrían aplicarse técnicas basadas en redes recurrentes, modelos temporales, filtros de suavizado o redes de grafos espacio-temporales. Estas técnicas permitirían analizar la evolución del esqueleto de forma más robusta y podrían mejorar la detección de inconsistencias en secuencias largas.

En definitiva, el proyecto desarrollado constituye una base sólida sobre la que continuar trabajando. Sus posibles ampliaciones pueden orientarse tanto a mejorar la comparación de modelos como a perfeccionar la aplicación software, ampliar los datasets utilizados y enriquecer los métodos de evaluación técnica. De esta forma, el trabajo puede evolucionar hacia una herramienta más completa para el análisis de modelos de detección de pose en vídeos 2D.

Bibliografía

- [1] Gary Bradski. Opencv: Open source computer vision library. *Dr. Dobb's Journal of Software Tools*, 2000.
- [2] Zhe Cao, Tomas Simon, Shih-En Wei, and Yaser Sheikh. Realtime multi-person 2d pose estimation using part affinity fields. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7291–7299, 2017.
- [3] Hao-Shu Fang, Shuqin Xie, Yu-Wing Tai, and Cewu Lu. Rmpe: Regional multi-person pose estimation. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2334–2343, 2017.
- [4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [5] Google AI Edge. Pose landmarker task guide. https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker, 2024.
- [6] Orla Hardiman, Ammar Al-Chalabi, Adriano Chio, Eileen M. Corr, Giancarlo Logroscino, Wim Robberecht, Pamela J. Shaw, Zachary Simmons, and Leonard H. van den Berg. Amyotrophic lateral sclerosis. *Nature Reviews Disease Primers*, 3:17071, 2017.
- [7] Or Hirschorn and Shai Avidan. Normalizing flows for human pose anomaly detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 13545–13554, 2023.
- [8] Tao Jiang, Peng Lu, Li Zhang, Ningsheng Ma, Rui Han, Chengqi Lyu, Yining Li, and Kai Chen. Rtmpose: Real-time multi-person pose estimation based on mmpose. *arXiv preprint arXiv:2303.07399*, 2023.

- [9] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.
- [10] Yann LeCun, L'eon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [11] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Doll'ar, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision*, pages 740–755. Springer, 2014.
- [12] Pegah Masrori and Philip Van Damme. Amyotrophic lateral sclerosis: A clinical review. *European Journal of Neurology*, 27(10):1918–1929, 2020.
- [13] Microsoft. Visual studio code documentation. <https://code.visualstudio.com/docs>, 2026.
- [14] Python Software Foundation. Python 3 documentation. <https://docs.python.org/3/>, 2026.
- [15] Ke Sun, Bin Xiao, Dong Liu, and Jingdong Wang. Deep high-resolution representation learning for human pose estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5693–5703, 2019.
- [16] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2 edition, 2022.
- [17] TensorFlow Hub. Movenet: Ultra fast and accurate pose detection model. <https://www.tensorflow.org/hub/tutorials/movenet>, 2024.
- [18] Alexander Toshev and Christian Szegedy. Deeppose: Human pose estimation via deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1653–1660, 2014.
- [19] Ultralytics. Pose estimation. <https://docs.ultralytics.com/tasks/pose/>, 2025.

- [20] Yufei Yu, Zhiyang Xu, Yanjie Zhao, Cewu Lu, and Jingdong Wang. Vitpose: Simple vision transformer baselines for human pose estimation. In *Advances in Neural Information Processing Systems*, 2022.